A Comprehensive Survey on LLM-Guided Evolutionary Search

Dongbin Zhao[a], Sicheng Li[a], Xiaobing Yang[a], Kaixuan Xu[a], Zhongyue Zhao[a], Binbin Zuo[a], Mingfei Wang[a], Zijie Zhao[a] and Yuanheng Zhu[a,*]

[a]Chinese Academy of Sciences Institute of Automation (CASIA), Beijing, China

ARTICLE INFO

Keywords:
large language models
evolutionary computation
LLM-guided evolutionary search
code evolution

ABSTRACT

Large Language Model-guided Evolutionary Search (LLM-ES), exemplified by AlphaEvolve, has recently demonstrated impressive performance in solving goal-driven problems with extremely large and complex search spaces. Unlike conventional LLM approaches based on single-shot or few-shot sampling, LLM-ES introduces population-based evolution to iteratively generate more novel and higher-quality solutions. Compared with traditional Evolutionary Computing algorithms, LLM-ES treats LLMs as semantic variation operators. This design extends the search space from low-dimensional vectors to expressive and language-defined structures such as code and programs. Meanwhile, the rich prior knowledge embedded in LLMs significantly improves the efficiency of evolutionary search in complex solution spaces. As a result, LLM-ES effectively exploits the test-time scaling capability of LLMs and achieves state-of-the-art performance in many open-ended problems. In this survey, we provide a systematic overview of this emerging field. We first formalize the problem setting and the general pipeline of LLM-ES. We then review existing works from several key perspectives, including population design, prompt builder, LLM model selection, evaluation design, and exploration-exploitation control. Finally, we summarize representative applications and discuss open challenges and future research directions.

1. Introduction

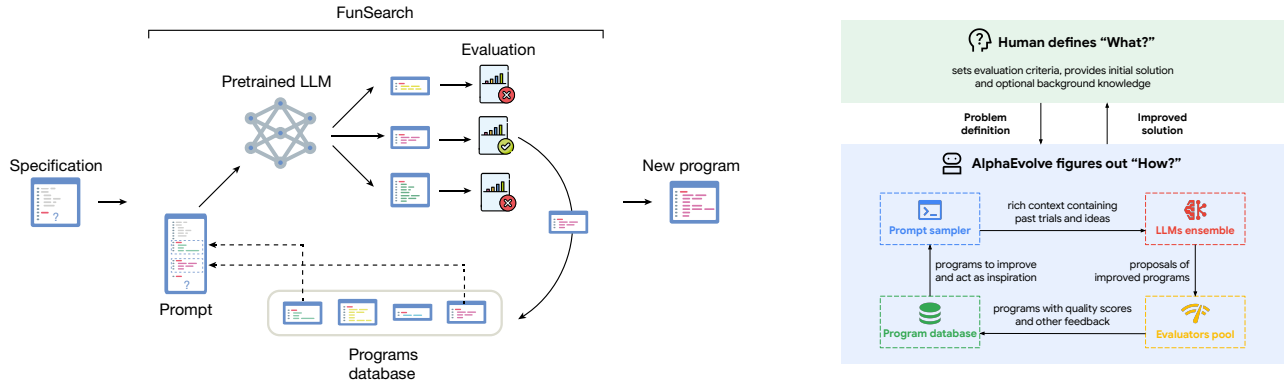
Large language models (LLMs) and vision-language models (VLMs) have shown strong capability across many application settings, including strategic games [147], scientific literature analysis [124, 131], robotics and embodied control [19, 20, 22, 47], and autonomous driving [170]. Many recent works also improve LLM capability through a wide range of training-time and inference-time methods [40, 129, 130]. However, open-ended problems remain difficult. When the desired answer is not already contained in training data, or when the solution space is too large for a single response to directly locate a high-quality result, one-shot generation becomes unreliable. This challenge is especially visible in scientific discovery, heuristic design, and new algorithm development, where progress often requires repeated search rather than isolated prediction.

Combining LLMs with evolutionary search has recently emerged as a promising paradigm for such open-ended optimization and discovery problems. FunSearch first showed that LLM-generated program variations, filtered by executable evaluation, can support mathematical discovery over code [115]. AlphaEvolve substantially generalized this closed loop to broader code artifacts, richer feedback, and wider scientific and algorithmic discovery settings [105]. Figure 1 shows representative overviews of these two foundational systems. Building on this foundation, later systems have already produced striking results across multiple domains: AlphaEvolve discovers the first improvement to 4×4 matrix multiplication after 56 years and also improves classical open constructions such as Erdős's minimum overlap problem and the 11-dimensional kissing number [105]; ThetaEvolve obtains new best-known bounds on open problems highlighted by AlphaEvolve, including circle packing and the first autocorrelation inequality [140]; and SR-SCIENTIST improves scientific equation-discovery baselines by absolute margins of 6%–35% across multiple scientific domains [144]. These results suggest that LLM-guided evolutionary search (LLM-ES) is no longer a collection of isolated demonstrations, but an emerging framework for long-horizon optimization over structured artifacts.

Subsequent systems expanded this line in multiple directions, including more structured research loops, sample-efficient program evolution, and adaptive search control [90, 140, 156]. The same paradigm has also begun to appear in applications such as architecture discovery, systems optimization, scientific equation discovery, and broader scientific modeling [46, 123, 144]. As a result, both the methodological design space and the range of target domains have grown rapidly. Figure 2 outlines the development timeline of LLM-ES from 2023 to 2026.

LLM-ES builds upon the iterative paradigm of traditional evolutionary computation (EC), while fundamentally extending how candidate solutions are generated and evolved. Traditional EC already provides the basic search backbone of initialization, evaluation, selection, variation, and replacement [37, 79, 127]. LLM-ES inherits this iterative structure, but changes the variation mechanism itself. Instead of relying only on hand-designed numerical or symbolic operators, it uses LLMs to generate, rewrite, patch, critique, or fuse structured artifacts such as code, heuristics, prompts, and workflows. This makes the search process far more semantically expressive, while also introducing new system-design questions about prompt construction, model usage, evaluation interfaces, search control, and maintained search state. Figure 3 compares the classical and LLM-guided evolutionary workflows.

*Corresponding author
ORCID(s):



(a) FunSearch [115] (b) AlphaEvolve [105]

Figure 1: Representative overviews of two foundational LLM-ES systems.

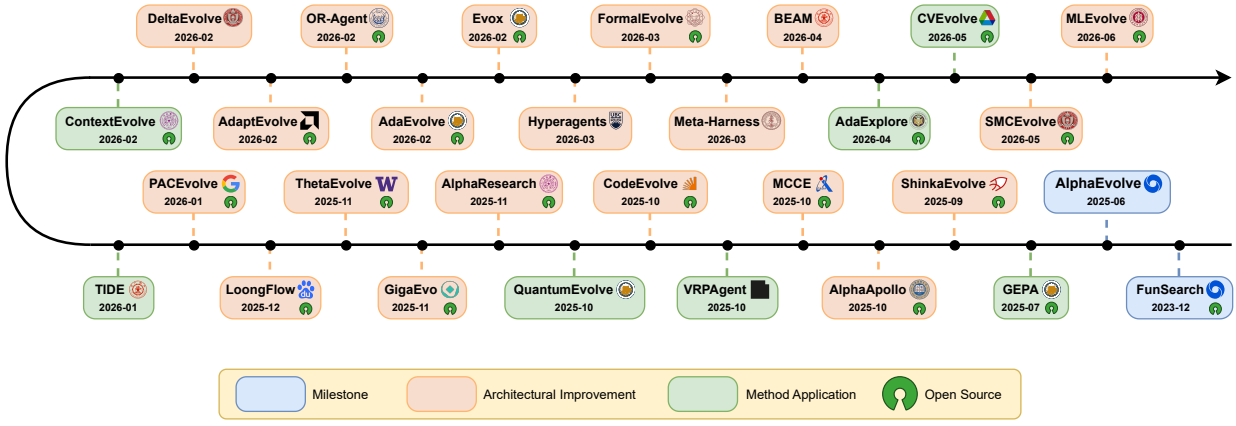
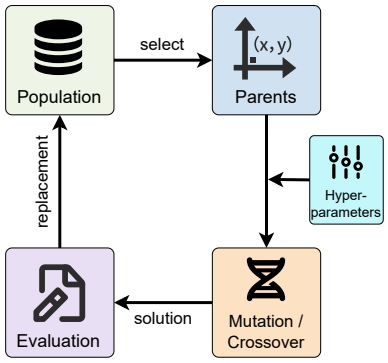


Figure 2: Development timeline of LLM-guided evolutionary search.

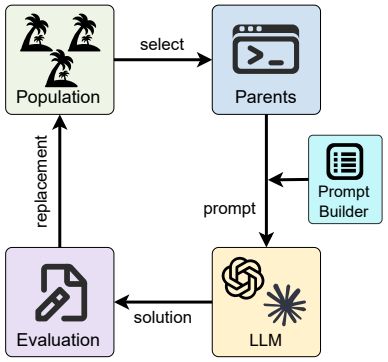
Before the emergence of LLM-ES, stronger LLM performance was often pursued through more capable generation procedures. One line of work improves a single response by eliciting better reasoning or searching over thought states, as in chain-of-thought prompting [141], self-consistency [138], and related reasoning-time search methods such as Tree of Thoughts [151]. Another line uses more complex multi-step workflows, such as tool-using agents in ReAct [152], self-reflection loops in Reflexion [120] and Self-Refine [95], or multi-agent interaction [35], to iteratively refine outputs within one task episode. These methods can be highly effective, but they usually seek the answer to a question through single or multiple reasoning paths within one episode. LLM-ES extends beyond this setting by introducing an external search loop in which candidate artifacts are repeatedly generated, evaluated, preserved, selected, and reused under longer-horizon search pressure.

Despite this fast progress, the field still lacks a dedicated survey. Existing papers usually emphasize domain breakthroughs or task-specific engineering choices. Related broader discussions mostly address adjacent areas instead, such as medical and healthcare LLMs [51, 146] and VLMs for robotics [47]. There are also broader surveys on the interplay between LLMs and evolutionary computation or evolutionary optimization [14, 143, 165]. However, these works do not systematically organize the design space of LLM-guided evolutionary search itself. To the best of our knowledge, this article is the first survey devoted specifically to LLM-ES. Our goal is therefore to organize current work around the key design dimensions that determine how an LLM-guided evolutionary system actually operates, while also summarizing its representative applications and resources.

The remainder of this paper is organized as follows. We begin from problem formulation, then move through the core layers of the evolutionary process, and finally review applications, open-source frameworks, and benchmarks. Within that middle part, population design provides the maintained search state, parent selection and candidate generation define how new candidates are produced from it, prompt construction integrates these search signals for model invocation, and evaluation closes the loop, while exploration-exploitation control acts across the search process as a cross-cutting design dimension. We conclude with open challenges and future directions.The organizational structure of this survey is visualized in Figure 4.



(a) Classical evolutionary search workflow



(b) LLM-guided evolutionary search workflow

Figure 3: Comparison of classical and LLM-guided evolutionary search workflows.

2. Problem Formulation of LLM-Guided Evolutionary Search

Like classical evolutionary computation (EC), LLM-guided evolutionary search solves an optimization problem by iteratively searching over a feasible space of candidate solutions. At a high level, the two paradigms share the same evolutionary logic: maintain candidate solutions, evaluate them under an objective, and use the evaluation results to guide the generation and retention of better candidates over time. However, LLM-guided systems differ substantially from classical EC in both the search space and the concrete search process. Instead of operating mainly on low-level numerical or symbolic encodings, LLM-ES searches over more expressive solution forms such as code, prompts, and workflows; instead of relying only on hand-designed mutation and crossover operators, they incorporate prompting, LLM generation, execution feedback, and memory or archive mechanisms into the optimization loop.

2.1. Optimization Search Problem Definition

The search problem can be formalized as an optimization problem over a feasible set:

$$x^* \in \arg\max_{x \in \mathcal{X}} F(x). \tag{1}$$

Here, $\mathcal{X}$ denotes the *search space*, namely the set of candidate solutions that satisfy the constraints of the task, and F denotes the *objective function*. In LLM-guided evolutionary search, the role of the evolutionary system is to propose new feasible candidates $x \in \mathcal{X}$ and use evaluator feedback to search for better solutions under F.

In both LLM-ES and traditional EC, the search problem can be divided into three categories according to the form of the objective function. The simplest case is *single-objective* optimization, where

$$F : \mathcal{X} \to \mathbb{R}, \tag{2}$$

so each candidate is assigned one scalar score and the search aims to maximize this value. This remains the default in many code and systems optimization settings such as *Magellan*, *KernelEvolve*, and SAT heuristic discovery [16, 84, 125]. A second regime is *multi-objective* optimization, where

$$F : \mathcal{X} \to \mathbb{R}^m, \tag{3}$$

and improvement is defined through a vector of competing criteria rather than one scalar score. In this case, selection is usually based on Pareto dominance rather than a total ordering [80, 98, 142]. A third regime is *quality-diversity* optimization. Here, the evaluator further exposes a behavior descriptor,

$$b : \mathcal{X} \to \mathcal{B}, \tag{4}$$

and the goal is not only to optimize $f(x)$ but also to populate different regions of behavior space $\mathcal{B}$ with high-quality elites [6, 31, 71]. In other words, single-objective search seeks one best solution, multi-objective search seeks a trade-off set, and QD search seeks a repertoire of behaviorally distinct high-performing solutions.

From the perspective of optimization objectives, the categorization of classical evolutionary search and LLM-guided evolutionary search is broadly similar. In contrast, the two paradigms differ much more substantially in the form of the evolving object. A systematic comparison of their evolving objects is presented in Table 1. In classical evolutionary optimization, the evolving object often takes the form of real-valued vectors, discrete sets, or symbolic structures. In the LLM-guided literature, the dominant case is an evolving object over *code*, where candidate solutions are executable programs, heuristics, or algorithmic components [46, 105, 125]. A broader class uses evolving objects over *code templates with tunable parameters*, where the template is evolved while the parameters are optimized separately by an additional procedure or optimizer [16, 80, 85]. Closely related are systems that evolve *executable generators*, namely

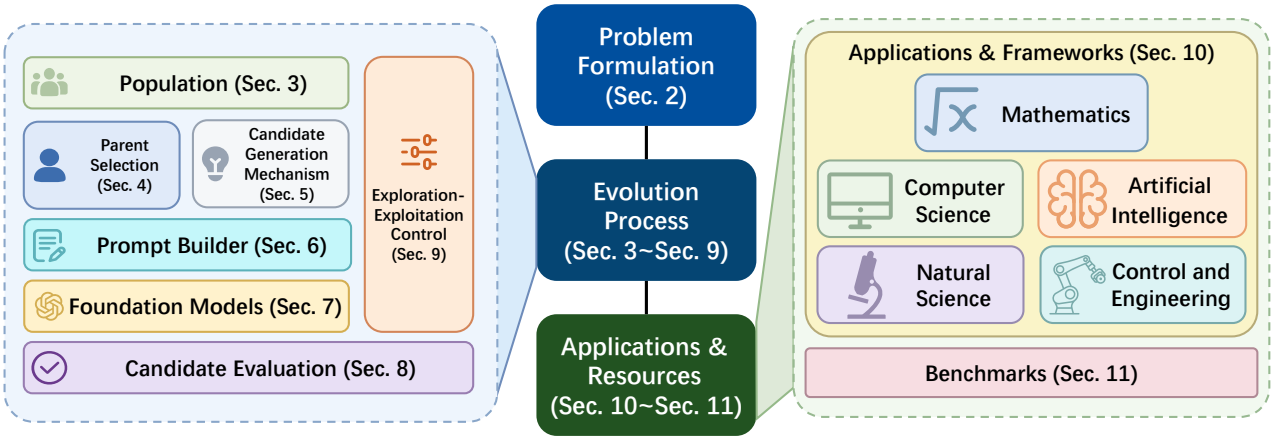


Figure 4: Organization of the survey.

higher-level executable artifacts which, like templates, are not themselves the final task solution but instead act as solution-producing generators [75, 157, 163]. Another class uses *structured solution abstractions* as the evolving object, where candidate solutions are represented through high-level structured descriptions whose components must be further realized into complete solutions [7, 53, 102, 145]. Other works use prompts, policies, or workflow structures as evolving objects, thereby moving beyond task solutions towards optimization process design [8, 12, 139].

2.2. Classical Evolutionary Search Workflow

For ease of comparison with LLM-guided evolutionary search, this subsection briefly reviews the main workflow of classical evolutionary computation (EC). More comprehensive overviews of EC can be found in [161].

Formally, the population at generation t can be written as a finite set of candidate solutions

$$P_t = \left\{ x_t^{(1)}, x_t^{(2)}, \dots, x_t^{(N)} \right\} \subseteq \mathcal{X}, \tag{5}$$

where N denotes the fixed population size. The evolutionary transition from P_t to P_{t+1} is characterized by a composite operator $\mathcal{T}$, which decomposes into three modular sub-operators:

$$P_{t+1} = \mathcal{T}(P_t) = \mathcal{R}\left(P_t, \mathcal{V}(\mathcal{S}(P_t))\right), \tag{6}$$

and the three operators are formally defined below.

Selection Operator $\mathcal{S}$: This operator takes the current population P_t as input and selects parent configurations A_t:

$$\mathcal{S}(P_t) = A_t. \tag{7}$$

For example, under fitness-proportionate selection, the sampling probability of any individual $x \in P_t$ is proportional to its objective value $F(x)$, which biases the search toward promising regions and forms the basis of exploitation in traditional EC.

Variation Operator $\mathcal{V}$: This operator takes the selected parents A_t as input, applies variation mechanisms to the selected parents, and produces an offspring set:

$$\mathcal{V}(A_t) = O_t. \tag{8}$$

Two common instances are mutation and crossover. Mutation introduces random perturbations to a parent, which is widely adopted in evolution strategies [50]:

$$x' \sim \mathcal{M}(x), \quad x' = x + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2 I), \tag{9}$$

Crossover recombines genetic information from two parents to produce novel offspring and is a core operation in GA and DE [54, 122]:

$$x' \sim C(x^{(i)}, x^{(j)}), \tag{10}$$

Compared with mutation, crossover can more effectively combine high-quality traits from different individuals.

Replacement Operator $\mathcal{R}$: This operator takes the tuple (P_t, O_t) as input and produces the next population through a replacement rule:

$$\mathcal{R}(P_t, O_t) = P_{t+1}. \tag{11}$$

The replacement mechanism determines how information from the current population and the offspring population is combined, and plays a central role in convergence control and elitism preservation [37].

From a distributional perspective, the classical evolutionary search workflow can be viewed as a density evolution process over $\mathcal{X}$. Let $p_t(x)$ denote the probability density of the population at generation t. Selection reweights the density by fitness, and variation acts as a diffusion kernel, forming a unified distributional optimization framework that is orthogonal to the semantic generation paradigm of LLMs.

2.3. LLM-Guided Evolutionary Search Workflow

In contrast to the hand-coded operator pipeline of traditional EC, LLM-guided evolutionary search redefines the variation mechanism via conditional semantic generation of large language models[105, 115]. It leverages the contextual

Table 1
Search space comparison between LLM-ES and conventional EC

(a) Classical Evolutionary Computation (EC)

Evolving object	Characteristics	Related work
Real-Valued Vectors	Continuous optimization domain; standard encoding for ES, DE and CMA-ES.	Evolution Strategies (ES) [61]; Differential Evolution (DE) [122]; CMA-ES [50]
Discrete Sets	Binary strings and combinatorial structures; for combinatorial optimization problems.	Genetic Algorithm (GA) [54]
Symbolic Structures	Tree-based representations (e.g., GP); evolves functional expressions and symbolic programs.	Genetic Programming (GP) [70]

(b) LLM-Guided Evolutionary Search

Evolving object	Characteristics	Related work
Code	Executable programs, heuristics, or algorithmic components.	AlphaEvolve [105]; ArchAgent [46]; SAT Heuristics Discovery [125]
Code Templates + Parameters	Template is evolved while parameters are optimized separately.	Magellan [16]; MADE-volve [80]; TIDE [15]
Executable Generators	The evolved artifact generates the final task solution through a fixed non-evolutionary execution procedure.	HyperAgents [163]; CyberE-volver [39]; Meta-Harness [75]
Structured Solution Abstractions	High-level structured abstractions of candidate solutions.	BEAM [145]; [7]; Archi-GenMS [102]
Workflow Structures	Moves beyond task solutions towards optimization process design.	GEPA [2]; Prompt-Bridge [139]; Evolving Excellence [8]

reasoning and code synthesis capabilities of LLMs to implement semantic-level evolution, which extends the search space to structured artifacts such as programs, prompts and algorithms.

The LLM-guided evolutionary loop can be formalized as a population-conditioned generative process. It replaces the traditional variation operator $\mathcal{V}$ with an LLM-driven variation operator $\mathcal{V}^{\Phi}$, where the prompt builder Φ conditions generation on population information[105]. The complete one-step evolutionary transition is defined as:

$$P_{t+1} = \mathcal{R}\big(P_t, \mathcal{V}^{\Phi}(\mathcal{S}(P_t))\big), \tag{12}$$

where $\mathcal{S}$ and $\mathcal{R}$ play the same roles as in the classical workflow.

LLM-Driven Variation Operator $\mathcal{V}^{\Phi}$: This operator maps the selected parent set to a generated offspring set through prompt-conditioned LLM generation:

$$\mathcal{V}^{\Phi}(P_t, A_t) = O_t. \tag{13}$$

The prompt builder Φ first maps the selected parent set to a prompt set,

$$Z_t = \Phi(A_t), \tag{14}$$

which compiles parent solutions and other relevant information into representations that bridge the evolutionary population state and the semantic generation space of LLMs[143]. The LLM then maps this prompt set to generated candidates in the structured search space $\mathcal{X}$ through semantic sampling,

$$O_t = \mathrm{LLM}(Z_t). \tag{15}$$

From a comparative perspective, the essential difference between the LLM-ES workflow and the classical evolutionary workflow lies in the variation mechanism: traditional EC relies on fixed syntactic perturbations, while LLM-guided search adopts adaptive semantic generation. This paradigm shift enables the framework to handle high-dimensional discrete structured search spaces such as program code, and naturally supports multi-objective optimization, quality-diversity search and cross-domain generalization [143].

3. Population

In LLM-guided evolutionary search, the population marks the current search frontier and provides promising parents for generating new candidates. In later search steps, the main editable solution and any auxiliary reference solutions are typically sampled from the maintained population through some selection mechanism. In this sense, population is a basic component of the LLM-ES workflow. Existing studies have explored this design space from several main aspects, including *population structures*, *elite archives*, and *replacement mechanisms*.

3.1. Population Structures

As evolution proceeds, increasingly many individuals are produced and evaluated. Newly generated individuals represent the current frontier of the search, while earlier historical individuals may preserve search branches that have not yet been fully explored but still contain potential value. For this reason, different algorithms maintain the population with different data structures. Figure 5 shows the main population structures existing in literatures.

3.1.1. Single-Individual Populations

The simplest population structure is a *single-individual* regime. Here each iteration may still generate multiple temporary candidates for comparison, but the system repeatedly

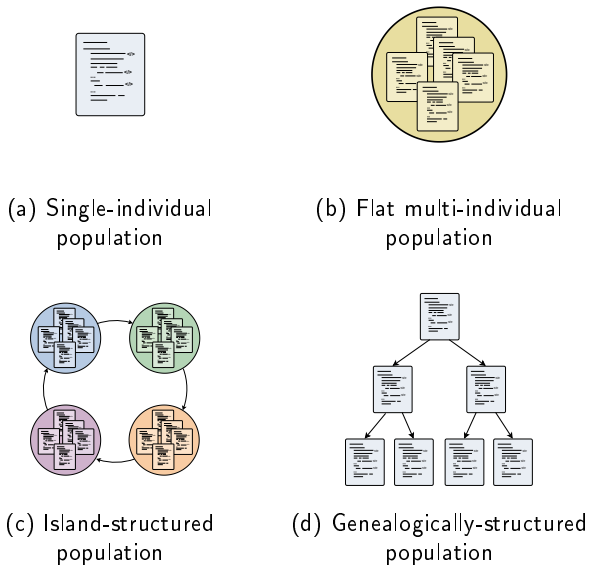


Figure 5: Representative population structures.

refines only one current candidate across iterations. [125] and [44] are representative examples: the search state remains centered on one incumbent artifact, with only better validated variants carried forward. This design is close to hill-climbing or $(1 + \lambda)$ search, and it is attractive when evaluation is expensive or when the main goal is iterative refinement of one artifact.

The main advantage of this population structure is its simplicity: storage, selection, and replacement are all straightforward, and search effort remains tightly focused on the current best direction. Its main limitation is the lack of persistent diversity, which makes the search more vulnerable to local optima and prevents potentially valuable historical branches from being retained for later reuse.

3.1.2. Flat Multi-Individual Populations

When only one individual is maintained, potentially useful historical branches are easily discarded. Storing multiple individuals makes it possible to preserve alternative search directions and reuse them later as parents or references for further generation. Therefore, a second regime stores *multiple individuals* in a flat form. The storage itself is unstructured, but it can be ordered according to the scores of the individuals. This population structure is very common in the current literature, with representative examples including AlphaResearch [156], ThetaEvolve [140], and X-evolve [160].

Some algorithms allow the individuals generated at each iteration to be continuously stored in the population. AlphaResearch [156], for example, is closer to an append-only candidate pool coupled with a best-so-far tracker. Others maintain a fixed population size, such as ThetaEvolve [140] and [73], which impose clearer survivor pressure through bounded ranked archives.

3.1.3. Island-Structured Populations

When a flat population is evolved as a whole, search may gradually concentrate on a narrow region and reduce the diversity of the maintained candidates. To mitigate this issue, many studies introduce island-structured populations as an explicit mechanism for diversity maintenance. The basic idea is to partition the population into multiple islands, let each island evolve mostly independently, and periodically allow information exchange across islands through migration or related coordination mechanisms.

ShinkaEvolve [74], TIDE [15], and [117] all distribute candidates across local subpopulations and then coordinate search through migration or reseeding. Compared with flat archives, this design makes diversity preservation a property of the population layout itself rather than an accidental byproduct of stochastic sampling.

Migration is introduced to let useful discoveries spread across islands without fully merging their local search trajectories. It therefore provides a mechanism for balancing local specialization and global information sharing. Existing studies mainly differ in two aspects. First, the migration schedule may be fixed or adaptive. AdaEvolve [11], MadEvolve [80], and AgentLTV [142] all adopt fixed migration intervals, while TIDE [15] triggers migration adaptively according to stagnation and structural similarity between islands. Second, the migrated object also varies across systems. Some methods transfer executable candidates directly, such as the top programs in AdaEvolve [11], high-fitness programs in AgentLTV [142], a fraction of the best members in MadEvolve [80], and non-best candidates in ShinkaEvolve [74]. TIDE [15] instead distinguishes between direct elite-code transfer and higher-level insight transfer. In topology, ring-based transfer is common, as in AdaEvolve [11] and MadEvolve [80].

3.1.4. Genealogically-Structured Populations

Genealogical structure is better viewed as an additional organizational layer rather than a standalone alternative to the population structures above. It does not conflict with flat or island-structured populations, and can be maintained together with them. In general, however, maintaining an explicit genealogical structure is most natural when the algorithm preserves relatively complete historical solutions instead of retaining only the current active population.

Some systems record only the parent-child relation between each child and its main editable candidate. In this case, the genealogical structure forms a tree [148] or, when multiple roots are maintained, a forest [168]. Other systems additionally record the relation between a child and its reference candidates. In that case, the resulting structure is no longer limited to a tree and becomes closer to a graph [33].

Genealogical structure is often combined with flat multi-individual populations [5, 33, 111], where the maintained genealogy helps organize branching search trajectories and guide which candidate should be expanded next. It can also be combined with island-structured population designs. In

Table 2
Comparison of elite archive styles.

Archive	What is kept	Characteristics	Papers
Single elite	One current best candidate	Simple, focused, low overhead but lacks diversity, prone to local optima, no history.	SAT heuristic discovery [125]; AlphaResearch [156]
Multiple elites	Several strong candidates in a ranked pool or archive	High-quality, balanced exploration-exploitation but limited diversity.	VRPAgent [55]; FM Agent [78]
QD elites	One elite per niche or descriptor cell	Structurally diverse but high overhead, requires descriptor functions.	CodeEvolve [4]; SEA-TS [148]; [114]

AgentLTV [142], the tree and the islands belong to different stages: a single MCTS tree is searched first, and its best program is then passed to multiple islands for later refinement. In CODEEVOLVE [4], each island maintains its own tree-structured solution lineages internally.

3.2. Elite Archives

Elites are candidate solutions with particularly strong, and often the best, observed performance under the current evaluation criterion. Their maintenance involves both updating the elite set when better candidates appear and preserving selected elites across subsequent search steps. In many LLM-ES systems, this functionality is handled by an *elite archive*, which forms a logical structure distinct from the main population even when the two may partially overlap in implementation. Maintaining such an archive is useful because it preserves historically important search information, especially the strongest solutions discovered so far and the search paths associated with them. A further point worth noting is that, under island-structured populations, elite archives can also implicitly support communication across islands. According to how many elites are stored and how the archive is maintained, existing designs can be categorized into *single-elite archives*, *multi-elite archives*, and *quality-diversity archives*. Table 2 summarizes these archive styles.

3.2.1. Single-Elite Archives

Single-elite maintenance is commonly used in hill-climbing systems or best-so-far tracking mechanisms embedded within larger search pipelines. The SAT heuristic-discovery framework is the clearest example, since only a strictly better executable offspring replaces the incumbent [125]. In such systems, the archive degenerates to one current best solution, and all evolutionary pressure is concentrated on improving or defending that incumbent.

Even when the broader system stores more information, a single-elite view may still be embedded inside a larger archive. AlphaResearch, for example, keeps a broader trajectory and candidate pool, but it also maintains an explicit best-so-far program that acts as the clearest elite summary of search progress [156].

3.2.2. Multi-Elite Archives

Multi-elite archives store several strong candidates rather than one winner. In this regime, the archive acts as a repository of high-quality solutions, so later search steps can draw on more than one promising direction at a time. Many systems in this category maintain a fixed-size top-k-style archive, including VRPAgent [55], FM Agent [78], and [73]. ShinkaEvolve and AdaEvolve instead maintain per-island archives of strong programs rather than one shared archive [11, 74]. This design supports both exploitation of strong candidates and reuse of alternative ideas without collapsing search to a single incumbent.

The concrete role of the archive can still vary across systems. In VRPAgent, the elite set is used directly for biased crossover and elite mutation [55]. In FM Agent, the elite pool guides later sampling and coordination [78]. In ThetaEvolve [140] and [73], the elite subset serves as the main source of reusable high-quality candidates for later iterations.

3.2.3. Quality-Diversity Archives

Conventional elite archives emphasize global quality, but may easily discard structurally different candidates that are not currently top-ranked. *Quality-diversity elites* instead maintain one elite per niche or cell defined by a descriptor function beyond the objective value itself. AlphaEvolve [105], CodeEvolve [4], SEA-TS [148], and [114] illustrate this pattern. Their importance lies in preserving heterogeneous solution modes, especially when a single global ranking would collapse the population into one narrow region of the search space.

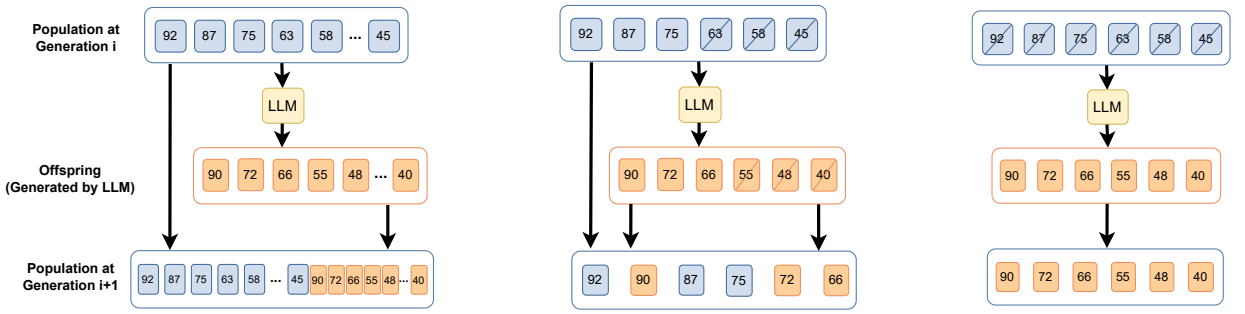
Compared with multi-island populations, quality-diversity archives maintain diversity through explicitly defined semantic niches or descriptors, instead of just different search branches.

3.3. Replacement Mechanisms

As iteration proceeds, more new candidates are produced and need to interact with the maintained population and archive. Accordingly, a suitable *replacement* mechanism is needed to update the population and archive over time. Common patterns include *incremental insertion*, *elitist replacement*, and *generational replacement*,whose core workflows are compared in Figure 6.

3.3.1. Incremental Insertion

Incremental insertion continuously adds newly generated candidates into a population or an archive, causing the reusable search history to grow over time. In this setting, selection pressure is shifted away from strict survivor filtering and toward later-stage parent selection or retrieval, which must identify promising prior candidates from an increasingly large population pool. AlphaResearch, Strategic Search, and X-evolve follow the direct append-only candidate-pool form [10, 156, 160]. SIDiffAgent is a slightly



(a) Incremental Insertion (b) Elitist Replacement (c) Generational Replacement

Figure 6: Three mainstream replacement mechanisms in LLM-based evolutionary computation

different variant: it incrementally stores compressed trajectory records rather than raw candidates, and these records still function as reusable search memory for later generation [41]. This mechanism preserves historical coverage and facilitates retrospective analysis.

3.3.2. Elitist Replacement

Elite replacement primarily governs updates to elite archives. It determines whether a newly evaluated candidate should replace or enter an existing elite entry according to the archive criterion, thereby maintaining a compact set of high-quality historical solutions for later reuse. This category includes top-k retention and MAP-Elites-style cell-wise replacement.

The choice of mechanism depends on the problem setting and the maintained population structure. ThetaEvolve [140], ShinkaEvolve [74], and AdaEvolve [11] all maintain bounded sets of strong candidates, in which replacement is performed in an individual-by-individual manner through top-k style retention. TIDE [15] applies a related idea at the batch level. By contrast, CodeEvolve [4] and SEA-TS [148] represent quality-diversity archives in a MAP-Elites style: a candidate is mapped into a descriptor cell and replaces the incumbent only when it is better within that cell.

3.3.3. Generational Replacement

Generational replacement updates the maintained population generation by generation using newly produced batches of candidates, rather than continuously accumulating candidates over time. In this regime, the current population serves as the parent pool for generating the next batch, after which it is replaced entirely by the newly formed generation while the previous population is discarded.

[166] is a typical example of pure population refresh: each iteration constructs a new candidate set from the current one, and only the refreshed set is carried forward to the next iteration. In contrast, [86] uses a more selective variant: it still refreshes the parent pool generation by generation, but only a filtered subset of the newly produced candidates is retained as the next generation.

4. Parent Selection

Populations and archives preserve the evolving search history of an LLM-ES system. However, only a small subset of these accumulated candidates can be reused when generating new offspring. In evolutionary computation, different parent-selection strategies induce different search biases and therefore directly influence the balance between broad exploration and focused refinement [161]. The same issue remains central in LLM-guided evolutionary systems, but the reused objects are often substantially more heterogeneous than in conventional EC. A framework may select candidates for direct inheritance, retrieve historical solutions as contextual references, or dynamically switch among multiple selection strategies depending on the current search stage and optimization objective.

Consequently, parent selection in LLM-ES is not only about choosing high-performing candidates, but also about deciding what forms of historical search information should be reused, how they should be retrieved, and how different selection mechanisms shape the overall search dynamics.

4.1. Selection Mechanisms

Existing methods can first be grouped by the kind of signal used in selection itself. Table 3 provides a compact overview. Some of these mechanisms are primarily used for choosing the main parent, some are more common for choosing auxiliary references, and some appear in both roles.

4.1.1. Uniform Selection

The simplest case is uniform selection, where a parent is chosen without explicit preference for stronger candidates, or with only a minimal structural restriction on the candidate pool. This includes purely random sampling as well as designs where the eligible pool is fixed in advance and the parent is drawn uniformly from the currently maintained candidate set.

Several systems contain this pattern, at least in one part of their search process. One form samples a single parent uniformly, as in AdaEvolve during exploration [11], [43], and the uniform branch of [77]. Another form samples multiple parents uniformly and uses them jointly, as in [25] and

Table 3
Representative parent-selection mechanisms in current LLM-ES systems.

Mechanism	Selection criterion	Role / advantage	Papers
Uniform selection	No explicit quality signal	Preserves broad exploration and avoids early concentration on a small elite set.	Simple Baselines [43]; AdaEvolve [11]; RSA [132]
Score-based selection	Raw score or fitness magnitude	Directly biases reuse toward strong candidates and gives simple exploitation pressure.	GRACE [118]; GigaEvo [69]; LoongFlow [133]
Rank-based selection	Relative ordering or elite subset membership	Avoids dependence on score scale and is more robust when raw scores are noisy or not directly comparable.	ThetaEvolve [140]; SR-SCIENTIST [144]; RankEvolve [103]
Search-statistic-based selection	UCT, PUCT, UCB, or related search utilities	Incorporates uncertainty, visit history, or frontier potential rather than only current solution quality.	SEA-TS [148]; AgentLTV [142]; TIDE [15]
Structure-based selection	Parent–child relations, ancestry paths, island membership, cell neighborhoods, or tree position	Preserves locality in the maintained search organization itself, especially along ancestry structure.	SEA-TS [148]; PACEvolve [149]; PhyloEvolve [168]
Evaluator-metadata-based selection	Failure types, counterexamples, diagnostics, or verifier records	Reuses informative evaluation outcomes beyond the final score alone.	Stateful Inference-Time Search [73]; [44]
Semantic-similarity-based selection	Query-dependent similarity or retrieval relevance	Resurfaces locally relevant historical exemplars or trajectories for the current search state.	ContextEvolve [123]; SIDiffAgent [41]; ASAP [30]

[132]. Such strategies emphasize broad exploration because they avoid early concentration on a small set of already strong solutions, but they also make relatively weak use of accumulated quality information.

4.1.2. Score-Based Selection

Score-based selection directly uses raw objective values to choose the next parent. Typical forms include proportional sampling, softmax sampling, roulette-wheel selection, and other mechanisms that depend on the numerical magnitude of the evaluation signal itself. Compared with uniform selection, score-based selection emphasizes exploitation because it biases reuse toward candidates that already achieve high measured performance.

Several systems contain this pattern. Under proportional sampling, the probability of selecting a candidate $x \in P_t$ can be written as

$$\mathbb{P}(x \mid P_t) = \frac{F(x)}{\sum_{x' \in P_t} F(x')}, \tag{16}$$

which is the basic form used by systems such as GRACE, ImprovEvolve, and GigaEvo [69, 71, 118]. A second common form is softmax sampling,

$$\mathbb{P}(x \mid P_t) = \frac{\exp(F(x)/\tau)}{\sum_{x' \in P_t} \exp(F(x')/\tau)}, \tag{17}$$

where τ is a temperature parameter, as in LoongFlow and related temperature-controlled variants [98, 133]. [72] uses stochastic universal sampling over fitness-weighted candidates. MCCE [113], REvolution [98], and Lark [126] also include fitness-based or roulette-wheel-style parent sampling.

4.1.3. Rank-Based Selection

Rank-based selection uses only the relative ordering among candidates rather than the raw magnitude of their scores. Compared with score-based selection, its main advantage is that it does not depend on the specific scale of the returned scores, and is therefore less sensitive to score calibration, rescaling, or task-dependent variation in score magnitude. Typical forms include top-k choice, percentile filtering, champion-challenger selection, bucketed ranking, and top/bottom curation.

Many LLM-ES systems use rank-based reuse. A common pattern is to first define a rank-based candidate subset $S(P_t)$, and then sample uniformly within that subset:

$$\mathbb{P}(x \mid P_t) = \begin{cases} \frac{1}{|S(P_t)|}, & x \in S(P_t), \\ 0, & \text{otherwise}, \end{cases} \tag{18}$$

where $S(P_t)$ may denote the top-k subset, a percentile-filtered subset, or another rank-defined elite set. A common pattern is that $S(P_t)$ is defined inside a maintained archive: ThetaEvolve [140] samples from a reranked program archive, SR-SCIENTIST [144] from the top-k equations in an experience archive, and PolicySmith and RankEvolve [36, 103] from top-performing solution archives. Other rank-based systems are less naturally summarized by this formula. For example, [154] uses a champion-challenger style repository, OPTAGENT directly passes the top-N queries to the next generation as parents [48], and [137] explicitly reuses top- and bottom-ranked historical functions as contrastive references. Rank-based choice is especially natural when raw scores are noisy, task-dependent, or not easily comparable across search stages, because it preserves

ordering information without overcommitting to score magnitude.

4.1.4. Search-Statistic-Based Selection

Search-statistic-based selection uses search-controller statistics rather than ordinary score or rank. Its main advantage is that it can incorporate signals beyond current solution quality. These signals may include uncertainty, visit history, estimated future potential, or learned utility scores derived from the search history itself. As a result, this mechanism is better suited to settings where the parent should be chosen as the next expansion point of an ongoing search process, rather than simply as the currently strongest candidate. Typical examples include UCT, PUCT, UCB-style utilities, frontier priorities, progress-aware scores, Thompson-sampling-style utilities, learned utility scores, and related tree or bandit statistics.

This pattern is particularly visible in tree- or frontier-based frameworks. For example, [99] and SEA-TS [148] both select parent nodes under a UCT-style tree policy, while AgentLTV [142] uses PUCT in its first stage. [77] extends this pattern in a different direction by learning a contrastive concept-level likelihood-ratio score from high- and low-performing programs and using this learned utility to reweight parent sampling. Related search-statistic-based choice also appears in TIDE [15], FELA [107], and [111], where parent selection is embedded into a broader decision process over branches, operators, or frontier actions. These designs illustrate that parent selection can itself be part of a learned or adaptive search controller rather than merely a survivor-reuse heuristic.

4.1.5. Structure-Based Selection

Structure-based selection chooses candidates according to how the population is organized, such as tree position, island membership, cell neighborhood, lineage position, or other structure-defined relations. Its main advantage is that it can preserve locality in the search organization itself, rather than relying only on score, rank, or generic search statistics.

This pattern appears more often in auxiliary-reference construction. Examples include SEA-TS [148], QuantEvolve [159], and CVEvolve [32], where references are drawn from nodes, lineages, or neighborhoods defined by the maintained population structure. The common idea is that structural proximity within the maintained search organization is itself treated as reusable search information.

4.1.6. Evaluator-Metadata-Based Selection

Evaluator-metadata-based selection chooses candidates according to metadata returned by the evaluator, such as failure types, counterexamples, diagnostic labels, verifier outputs, or other attached records. Its advantage is that it makes reuse directly responsive to what recent evaluation has revealed, so later generation can be conditioned not only on prior successes but also on informative mistakes.

This mechanism also appears primarily in reference selection. Representative cases include [73], [23] and [44], where evaluator-derived failures, diagnostics, or counterexamples are selectively resurfaced for later search steps.

4.1.7. Semantic-Similarity-Based Selection

Semantic-similarity-based selection chooses candidates by semantic relevance, similarity matching, or related memory lookup. Its main advantage is that it can resurface historically useful context in a query-dependent way, instead of relying only on global score, rank, or fixed structural relations.

This mechanism is currently more visible in auxiliary-reference reuse. In such cases, the query is usually formed from the current search state itself, such as the current task instance, the selected parent candidate, the current partial solution, or the latest trajectory context. Representative examples include ContextEvolve [123] and SIDiffAgent [41], where historical exemplars or prior trajectories are retrieved by similarity to the current context, and ASAP [30], where related historical optimization cases are retrieved for similar model and hardware settings.

4.2. Selection-Mechanism Control

When multiple selection mechanisms coexist in the same framework, an additional design question arises: what determines which mechanism is used at the current step? Existing systems mainly address this issue in two ways. Some dynamically switch among selection mechanisms according to the current search regime, most commonly to balance exploration and exploitation. Others assign different mechanisms to different selection roles, such as choosing primary parents versus retrieving auxiliary references.

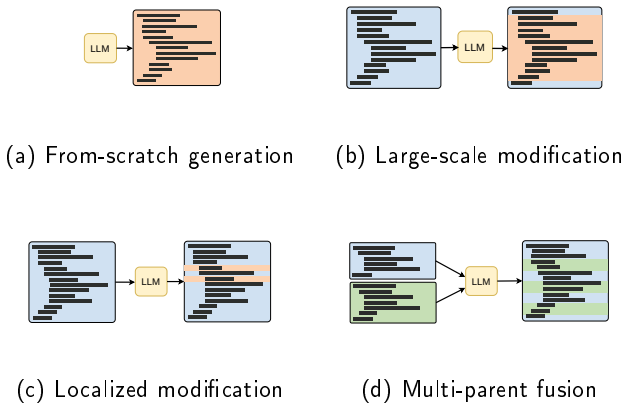
4.2.1. Mode-Controlled Selection-Mechanism Control

The first pattern is to let the active selection mechanism depend on the current search mode. In such systems, the selection target may remain the same, but the system changes how that target is sampled when it wants broader exploration or more focused exploitation.

This pattern is especially clear in AdaEvolve and RankEvolve. AdaEvolve uses different main-parent sampling rules under exploration and exploitation, switching between broader local-archive sampling and stronger fitness-biased reuse [11]. RankEvolve likewise alternates among several parent-selection strategies, including uniform, elite-oriented, and weighted sampling from the current island [103]. The main advantage of this design is that it allows one framework to combine the robustness of broad sampling with the efficiency of stronger selective pressure, rather than fixing one parent-selection bias for the entire run.

4.2.2. Object-Specific Selection-Mechanism Control

The second pattern is to let the active mechanism depend on the reuse role that the selected candidate will play. This is particularly important in LLM-ES systems because the prompt often reuses historical candidates in more than one way. Within a single iteration, the main editable parent, auxiliary reference solutions, retrieved search trajectories,



(a) From-scratch generation (b) Large-scale modification

(c) Localized modification (d) Multi-parent fusion

Figure 7: Candidate generation mechanisms.

and evaluator-derived diagnostic cases may each be selected using different mechanisms.

In current systems, this logic often takes the form of selecting the main editable candidate through one mechanism and auxiliary references through another. This design appears in [137], CODEEVOLVE [4], and SEA-TS [148]. The main advantage is that it lets the system match the selection mechanism to the role of the reused object.

Some systems combine both forms of control. AdaEvolve is one example because it distinguishes main parents from inspirations while also changing parent-selection behavior across exploration and exploitation modes [11]. More structurally special systems such as X-evolve also mix different control logics across different reusable objects and different levels of the search process [160].

5. Candidate Generation Mechanism

After parent candidates are selected, the search process proceeds to decide how the next candidate should be generated from them. This is analogous to classical EC, where different operators determine the number of parents involved and the relation between parents and offspring. In LLM-ES, the corresponding decision is realized through prompt-conditioned LLM calls rather than through hand-coded variation operators.

5.1. Candidate Generation Mechanisms

The generation mechanism determines whether the offspring is produced from scratch, by broadly rewriting one parent, by locally editing one parent, or by fusing multiple parents. These choices define the effective parent cardinality and the degree of continuity between parent and offspring. As summarized in Figure 7, four common mechanisms appear in the current literature: from-scratch generation, large-scale modification, localized modification, and multi-parent fusion.

5.1.1. From-Scratch Generation

The simplest generation mechanism is *from-scratch generation*. Under this mechanism, the prompt asks the LLM to propose a new candidate directly from the task description, retrieved knowledge, or initialization context, without editing any candidate parent. The generation therefore relies mainly on the LLM's background knowledge together with human prior guidance embedded in the prompt. PopulationEvolve [166] and QuantEvolve [159] are representative examples. Under this generation style, the effective parent cardinality is usually zero. Some systems may still expose retrieved examples or domain heuristics, but these function as prompt priors rather than genealogical parents [30, 166].

5.1.2. Large-Scale Modification

A second family uses the prompt to request a *large-scale modification* of an existing solution. This mechanism uses an existing solution as the basis for generation, but only preserves a limited portion of its content and allows broad revisions at the level of an entire function, algorithm, policy, or other structured artifact. This mechanism gives the LLM more freedom to propose substantial conceptual changes while still retaining some continuity with the current solution. [42], DeltaEvolve [63] and AdaEvolve [11] all exemplify this pattern.

The most common form is unary rewrite: one parent is selected and broadly rewritten into a descendant. [42], ThetaEvolve [140], and AdaEvolve [11] all follow this pattern, where a selected parent is revised at a relatively coarse granularity rather than edited through local patches. Some systems additionally introduce other reference solutions besides the rewritten parent. DeltaEvolve [63], EvolveSignal [136], and EvoSLD [85] are useful examples.

Because the resulting offspring may differ substantially from the parent, this mechanism tends to preserve less local continuity during search. As a result, its exploit capability is similar to from-scratch generation and is relatively weak.

5.1.3. Localized Modification

A third family uses the prompt to request a *localized modification* of the parent through patches, search/replace blocks, explicit diffs, or other tightly scoped edits. Compared with large-scale modification, this mechanism preserves more of the parent and constrains the generated change to a smaller region, which makes the search process easier to control. It is therefore better suited to targeted improvement and more stable refinement. AlphaEvolve [105] is the canonical case, since its operator is grounded in structured SEARCH/REPLACE edits rather than unconstrained regeneration. CodeEvolve [4], RankEvolve [103], and GigaEvo [69] follow the same broad logic, and ShinkaEvolve [74] includes a comparable diff-edit mode within a broader hybrid framework.

This family is again predominantly unary in genealogy: one parent is directly edited, even when several archived references are shown alongside it. AlphaEvolve augments the editable target with inspiration programs from its database

[105]. CodeEvolve adds nearby ancestors and inspiration solutions around the target parent [4]. RankEvolve further enriches the prompt with additional contextual references from an island-based MAP-Elites setting [103].

The limitation of localized modification is that the search can become overly concentrated around the current parent. Small edits are less suitable for structural rewrites or larger conceptual changes, and repeated local refinement may make it harder to escape a nearby local optimum. As a result, this mechanism often needs to be combined with broader generation modes or more diverse parent selection to maintain sufficient exploration.

5.1.4. Multi-Parent Fusion

The fourth family performs *fusion*, namely synthesizing a child by combining information from multiple parent candidates. The advantage of this mechanism is that it can combine complementary information from different parents, rather than refining only one existing search branch. It is therefore more suitable for cross-branch recombination and for constructing offspring that inherits useful traits from multiple prior solutions. VRPAgent [55], [52], and [25] all implement explicit multi-parent combination. ShinkaEvolve [74] likewise includes a crossover mutation mode, and TIDE [15] provides recombination-style operators inside a broader operator portfolio.

Fusion can be implemented at different levels. At the code or structure level, VRPAgent uses crossover to directly recombine heuristic operators within a fixed metaheuristic framework [55]. At a more semantic level, the adversarial imitation-learning work prompts the LLM to combine and refine parent programs according to their structural and behavioral properties, so the offspring is generated through semantic recomposition rather than explicit code splicing [25]. Pareto-Grid-Guided search further inserts reflective feedback before fusion, so parent heuristics are first analyzed and then recombined under additional guidance [52]. TIDE extends this line further by separating exploratory crossover from backbone extraction, and by reusing fusion as a reset mechanism that reconstructs a high-quality seed from accumulated search experience [15].

5.2. Generation Mechanism Selection

The four generation mechanisms discussed above each have their own strengths and limitations. Using different mechanisms at different stages of search can therefore help exploit their respective advantages. Some systems accordingly combine multiple generation mechanisms within their evolutionary process. This in turn raises the question of when to use which mechanism. Especially when computational budget or LLM calling budget is limited, the mechanism selection strategy largely determines the evolutionary behavior of LLM-ES. Different works design different selection strategies for this purpose.

5.2.1. Fixed Selection

The simplest design is to fix the use of generation mechanisms in advance, either through explicit probabilities over an operator portfolio or through a predetermined operator pipeline. ShinkaEvolve directly specifies probability vectors over diff edits, full rewrites, and crossover mutation [74]. MadEvolve follows the same pattern with a configurable mix of differential patches, full rewrites, and synthesis [80]. [52] likewise keeps a fixed mutation–crossover split, with semantic clustering only constraining crossover pairing. VRPAgent is fixed at the pipeline level: crossover generates offspring, while mutation is reserved for elite refinement [55].

The limitation of fixed selection is that the allocation is decided before the run and remains unchanged throughout search. As a result, it cannot react when different generation mechanisms become more or less useful at different search stages, and may waste budget on mechanisms that are less effective.

5.2.2. Scheduled Selection

Scheduled selection assigns different generation mechanisms to different stages of the evolutionary process. AgentLTV is a representative case. It follows a two-stage search process, where the MCTS stage mainly expands the search tree through mutation-style candidate edits on existing programs, while the subsequent evolutionary algorithm stage introduces a evolutionary process with crossover and mutation [142]. A similar stage-wise design appears in [137], where early iterations reduce reuse of historical candidates to encourage broader exploratory generation, and later iterations reintroduce prior candidates to support more refinement-oriented generation.

5.2.3. Adaptive Selection

Scheduled selection requires the stages to be predefined in advance, which relies on human prior knowledge and is relatively less generalizable across different problems. Adaptive selection instead updates the choice of generation mechanism online according to the observed search process.

Representative examples include [10], where a Strategist decides at each iteration whether the Implementor should *refine*, *combine*, or *innovate*; TIDE, which uses an explicit operator menu with two crossover-style strategies (`E1`, `E2`) and three mutation-style strategies (`M1`, `M2`, `M3`) and selects among them with UCB [15]; and REvolution, which maintains a strategy menu whose exact contents depend on whether the current subpopulation contains failed or successful designs, and updates strategy choice by a hybrid UCB-Softmax rule [98]. In general, adaptive selection bases this decision on observed search feedback, such as recent operator outcomes, search progress, or the current state of the maintained population. Its goal is to allocate the available generation mechanisms more appropriately to the current search regime, thereby improving the balance between exploration and refinement.

6. Prompt Builder

After parent candidates and candidate generation mechanisms have been determined, LLM-ES still needs to organize

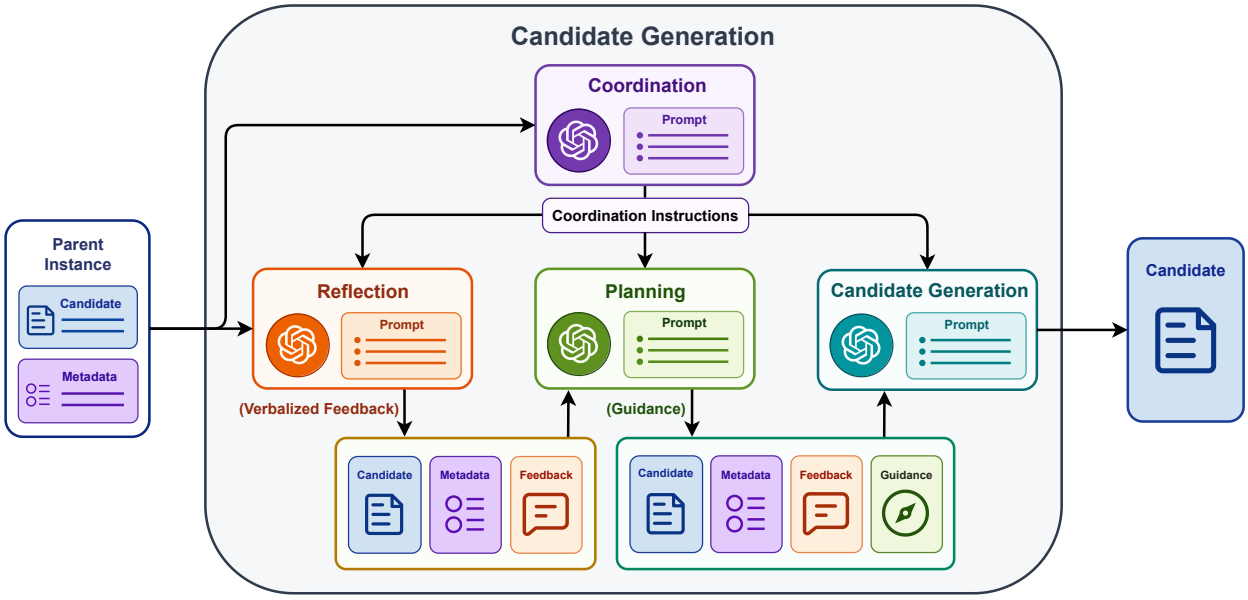


Figure 8: Multi-agent candidate generation workflow

them together with task information into the actual prompt consumed by the LLM. This is the role of the *prompt builder*. It determines what task information is visible, how selected parent context is embedded, how generation intent is expressed, and how additional prior solutions are presented as references.

The prompt builder is realized through prompt templates, namely concrete textual scaffolds that organize the relevant information into the input presented to the LLM. Although the detailed contents of these templates vary substantially across systems, templates intended to elicit the same type of output often share a similar structure. In existing multi-agent LLM-ES systems, these different output types are typically associated with different *roles*. Such roles can be divided into *coordination*, *planning*, *candidate generation*, and *reflection*, which in turn correspond to different prompt structures. Figure 8 illustrates a representative workflow in which these roles interact and iteratively exchange information to produce candidate solutions. We therefore organize the discussion mainly through the major prompt structures associated with these roles.

6.1. Candidate-Generation Prompt Structures

Candidate-generation prompts are the main interface through which LLM-ES produces new candidate solutions. In multi-agent settings, they are commonly associated with *candidate generation roles*. Their structure typically combines a small set of mandatory sections with optional contextual sections. Figure 9 shows a representative candidate-generation prompt structure.

In such structures, the mandatory sections consist of *task description*, *code interface*, and *response format*. The *task description* specifies the optimization target and the evaluation requirements. The *code interface* defines the executable form of the candidate to be generated or revised, including its required inputs, outputs, and functional structure. The *response format* specifies how the LLM should present the returned candidate. A representative example appears in [137]. Its prompt specifies the optimization target through a task-description block, constrains the editable candidate through a code-interface block, and fixes the LLM output through a response-format block.

Besides these required sections, candidate-generation prompts often include several optional blocks. Unless the candidate is generated from scratch, the prompt needs to provide a main parent candidate. This candidate serves as the base object to be evolved or revised, and it is placed in the *editable artifact* block. Another optional block is *auxiliary context*, which contains solution-related information that is not itself the editable target. Its main role is to provide additional search information around the current generation step. This may take the form of prior solutions, retrieved exemplars, historical records, hardware specifications, profiler outputs, or previous feedback [18, 105, 142]. *Constraints and guidance* can specialize the generic task statement into domain-specific feasibility rules [99], encode implementation constraints or operational guidance [43, 72] or add directional instructions [11, 18, 105]. By injecting human prior knowledge into the prompt, this block can narrow the effective search region and improve search efficiency.

6.2. Reflective and Meta-Guidance Prompt Structures

Direct candidate generation may be unstable, may provide limited control over search direction, and often relies on pre-defined human guidance, which motivates an intermediate step in which the LLM first generates reflection or guidance before subsequent candidate generation. These

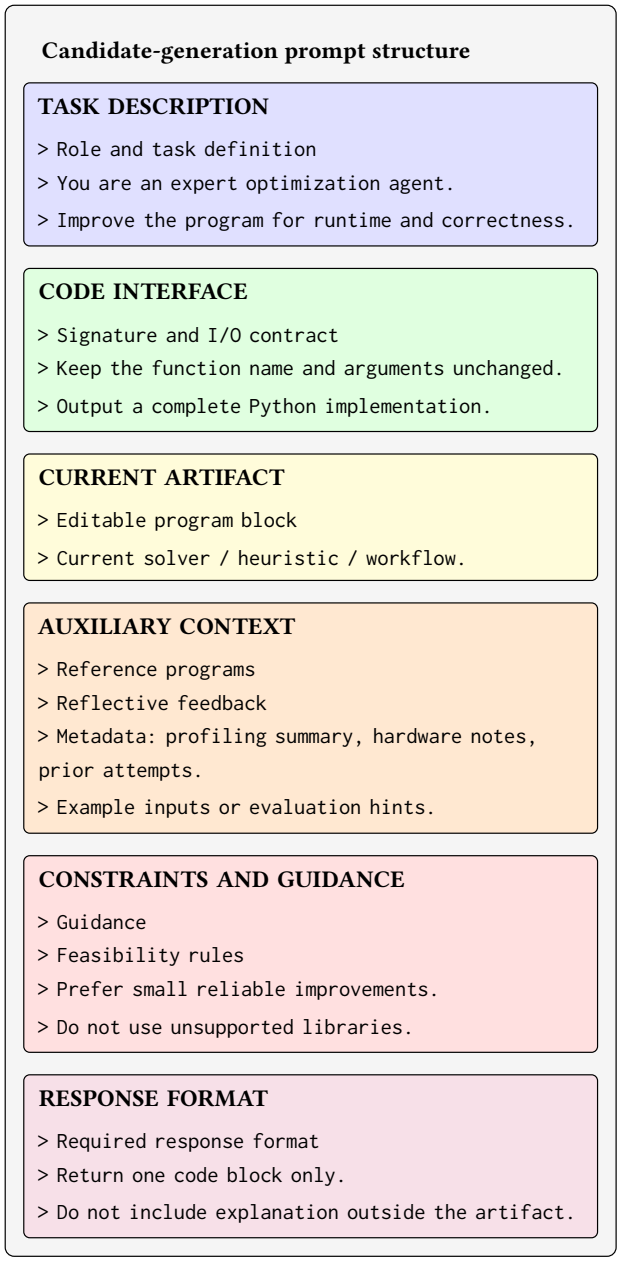


Figure 9: Candidate-generation prompt struture.

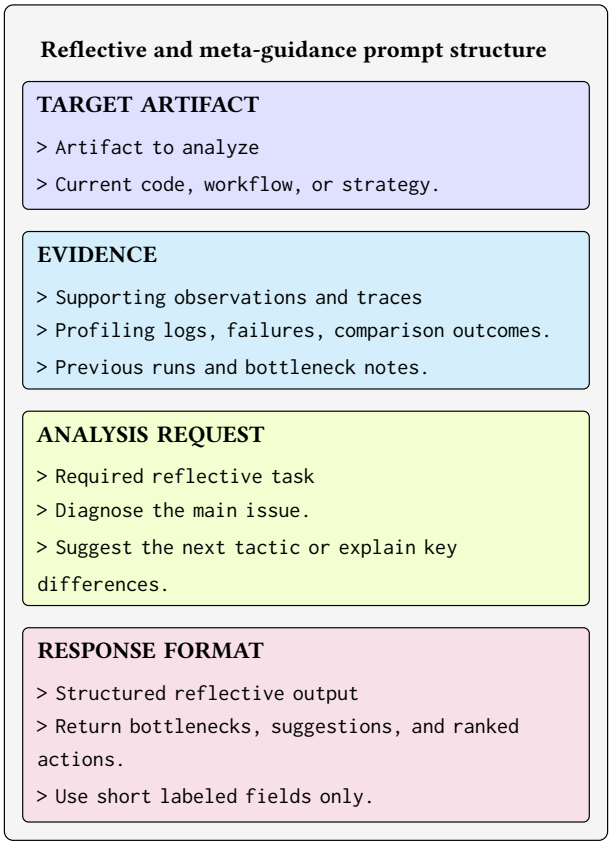


Figure 10: Reflective and meta-guidance prompt structure.

prompts are commonly used when the search loop needs intermediate analysis or higher-level directional guidance before the next candidate is produced. In multi-agent settings, reflective prompts are mainly associated with *reflection roles*, whereas meta-guidance prompts are mainly associated with *planning roles*. Figure 10 illustrates a representative reflective or meta-guidance prompt structure. Such prompts are usually organized around four sections: a *target artifact* that specifies the object under analysis, *evidence* that provides supporting observations or traces, an *analysis request* that states what the model should infer or recommend, and a *response format* that constrains the returned output. Across both reflective and meta-guidance prompts, this shared scaffold remains the same, while the functional difference mainly lies in the specific content of the *analysis request*.

Reflective prompts analyze current candidates and their observed outcomes. These prompts usually include an artifact under analysis, some evidence such as evaluation traces, logs, comparison pairs, or previous runs, and a request for structured diagnosis or reflection. ASAP is a clear example: its analyzer prompt consumes experiment metadata and profiling outputs and produces a structured bottleneck report [30]. AgentLTV likewise includes prompts that compare two code solutions, explain their methodological differences, or synthesize a hybrid strategy from the comparison [142]. GEPA [2] provide closely related cases in which the prompt is designed to return evaluative feedback or ratings.

Meta-guidance prompts determine how the search should proceed next. Reflective prompts guide the LLM to output judgments about the quality and shortcomings of a solution or modification itself, whereas meta-guidance prompts guide the LLM to output guidance for future evolutionary directions. AdaEvolve is the clearest example because it explicitly prompts the model to generate breakthrough tactics conditioned on the evaluator, the current best program, and prior unsuccessful tactics [11]. AgentLTV also contains prompts that select an advice type or format a free-form idea into a structured method description, while cuPilot uses prompts to

generate or translate optimization strategies for later kernel evolution [18, 142].

The outputs of reflective and meta-guidance prompts are usually stored together with the corresponding candidate as reusable search information. When that candidate is retrieved later, the associated reflection and guidance can be retrieved together with it and integrated into subsequent prompts. One representative use appears in candidate-generation steps, where reflective feedback can be placed in the *auxiliary context* block, while directional guidance can be placed in the *constraints and guidance* block of the candidate-generation prompt structure shown in Figure 9.

6.3. Coordination-Oriented Prompting

Coordination-oriented prompts manage the overall search process by routing information across prompts, dispatching subtasks, maintaining persistent state, and organizing interactions among other components of the workflow. In multi-agent settings, these prompts are typically associated with *coordination roles*. Unlike candidate-generation and reflective prompts, however, current papers usually describe them more clearly in terms of workflow function than in terms of stable reusable section-level templates.

ASAP provides a representative *Coordinator Agent* that mediates between the user, the *Analyzer Agent*, and the *Proposal Agent* [30]. [73] similarly uses a controller to manage persistent state and coordinate interactions among specialized roles. OR-Agent further instantiates an *OR Agent* for global coordination [89]. In cuPilot, the *SCE Manager* plays a related coordination role over strategy initialization and crossover [18].

7. Foundation Models

After the prompt is prepared by the prompt builder, it is passed to an LLM to produce a response. Today, thousands of foundation models are available. Owing to differences in model architecture, training paradigm, and service form, these models vary substantially in both capability and cost. For example, GPT/o-series models are often used for strong coding, repair, and difficult-case reasoning, Gemini models are frequently used for high-throughput exploration or multimodal synthesis, and open families such as DeepSeek, Qwen, and Llama are often adopted in self-hosted or cost-sensitive settings. Table 4 summarizes the foundation-model families that recur most frequently in surveyed LLM-ES systems. Existing works can be categorized according to the number of models they use, namely single-model settings and multi-model settings. Figure 11 summarizes the common model-invocation patterns under these settings.

7.1. Single-Model Setting

The most straightforward setting is to employ the same base model throughout the entire optimization pipeline. This paradigm is simple to implement and introduces relatively few additional experimental variables. As a result, it has been widely adopted in many studies on LLM-based optimization and LLM-driven evolution, including OPRO [150], FunSearch [115], and Eureka [94]. Because the same foundation model can still exhibit nontrivial performance differences under different inference configurations, some systems use this property to increase usage diversity even in a single-model setting. For example, EvoCAD uses a lower temperature for evaluation tasks such as visual description and ranking, but a higher temperature for generative operations such as initialization, crossover, and mutation [110].

Methodologically, the main attraction of the single-model setting is control. It reduces confounding variance from heterogeneous model APIs and makes it easier to attribute gains to prompt design, population management, or evaluation design. Its main limitation is limited flexibility: the same model must handle all search stages, so the system cannot easily assign lower-cost models to simple subtasks and more capable models to more difficult ones.

7.2. Multi-Model Setting

Some LLM-ES systems introduce multiple foundation models in order to exploit their respective performance advantages. In such case, the system must decide how these models are allocated during search. We distinguish between *static assignment* and *dynamic assignment*.

7.2.1. Static Assignment

In static assignment, multiple models may appear in the same system, but their usage pattern remains fixed throughout the evolutionary process. Most such systems use role-based assignment, where models are bound to fixed generator, refinement, review, transfer, or team roles, as in ArchAgent [46] and PromptBridge [139]. A less common variant is fixed-probability assignment: Applied AlphaEvolve allocates model usage through a static mixture rather than through run-time reassignment [6]. The main advantage of static assignment is simplicity: the roles of different models are easy to specify and interpret, and the resulting system behavior is relatively stable.

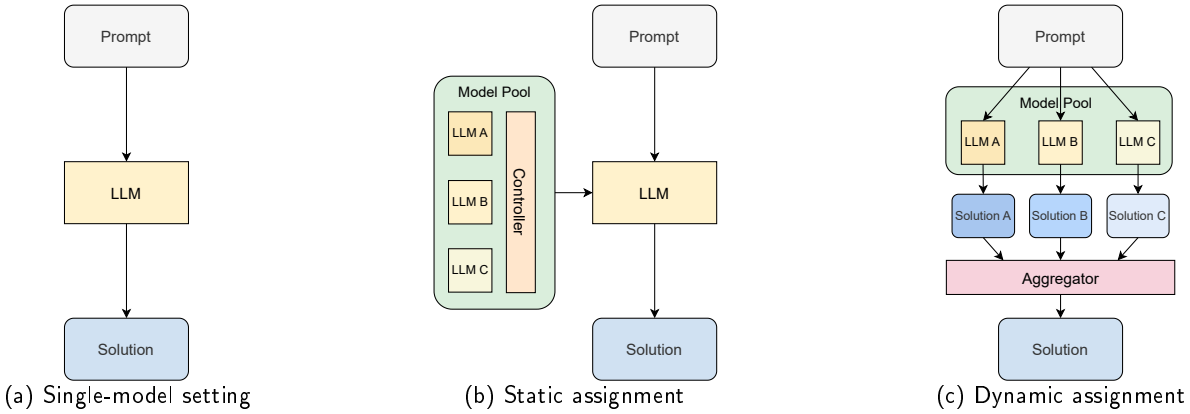
7.2.2. Dynamic Assignment

Dynamic assignment addresses the main limitation of static assignment: a fixed allocation policy cannot react to stage-dependent search conditions. By changing model usage across stages, dynamic assignment can better match appropriate models to the current needs of the search process. The trade-off is a more elaborate control policy and additional system complexity.

The cleanest example is AdaptEvolve, where each refinement step starts from a small model and then uses token-level uncertainty features to decide whether the current candidate should be kept or escalated to a larger model; the routing policy itself is further updated online to adapt to the non-stationary evolutionary process [114]. At the generation level, MCCE alternates a frozen API model and a trainable local model as evolutionary operators across generations, so the active model changes with the outer co-evolutionary process [113]. ShinkaEvolve instead updates LLM sampling probabilities after each generation using a bandit-style rule

Table 4
Foundation models appearing in LLM-ES literatures.

Model family	Open-source	Representative properties	Common strengths / typical role	Papers
OpenAI GPT / o-series	No	Proprietary API family; strong coding and general reasoning; some variants are multimodal; o-series emphasizes reasoning	High-quality generation, repair, reflection, and difficult-case reasoning	AgentLTV [142]; [137]; AdaEvolve [11]; CodeEvolve [4]
Google Gemini	No	Proprietary multimodal family; long-context support; Flash variants emphasize speed, Pro variants emphasize capability	High-throughput exploration with Flash; stronger synthesis and refinement with Pro models	AlphaEvolve [105]; ASAP [30]; ArchAgent [46]; [99]
Anthropic Claude	No	Proprietary long-context family; strong coding and instruction following	Code generation and refinement; strong ensemble member or fallback model	AgentLTV [142]; CodeEvolve [4]; [56]
DeepSeek	Yes	Family spanning general and reasoning-oriented variants; strong coding and mathematical reasoning	Backbone for code evolution and reasoning-heavy search	[125]; CUDA-L1 [81]; [56]
Qwen	Yes	Multilingual family with broad size range; coder variants are common in program evolution	Cost-efficient backbone for self-hosted coding and small/large model pairing	AdaptEvolve [114]; CodeEvolve [4]; Applied AlphaEvolve [6]
Llama	Yes	Instruction-tuned family; widely deployable locally; common strong baseline	Self-hosted or baseline backbone; useful for comparison and lower-cost deployment	[73]; [137]
Gemma	Yes	Compact family with lightweight deployment options; newer variants also support multimodal use	Lightweight comparison model for efficiency-conscious settings	[73]; DUEL-EVOLVE [66]



(a) Single-model setting (b) Static assignment (c) Dynamic assignment

Figure 11: Common model-invocation patterns in LLM-guided evolutionary systems.

based on relative improvement, reallocating future model usage according to observed search performance [74].

8. Candidate Evaluation

After a new candidate is generated by the LLM, its quality must be evaluated to determine whether it should be retained in the population or archive. In most LLM-ES systems, the problem designer provides an evaluator function, whose direct execution returns a scalar objective or a multi-objective vector. This default object evaluation is an implicit convention in the literature. A small number of systems instead use preference, ranking, or other comparative judgments as the evaluation signal, which is discussed first in this section. Beyond the primary signal itself, the discussion then focuses on two additional aspects that can arise under different evaluation interfaces: extra feedback returned together with the evaluation result, and evaluation procedures for time-consuming, expensive, noisy, or fragile evaluators.The evaluation output structure of LLM-ES systems is shown in Figure 12.

8.1. Preference-Based Evaluation

Some systems depart from the default scalar-score interface and instead treat *preference or ranking judgments* as the primary evaluation signal. In this regime, evaluation does not begin by assigning each candidate an absolute numeric score and then attaching auxiliary relational information. Rather, pairwise preferences, ranked lists, or other comparative judgments are the core output of the evaluator, and any

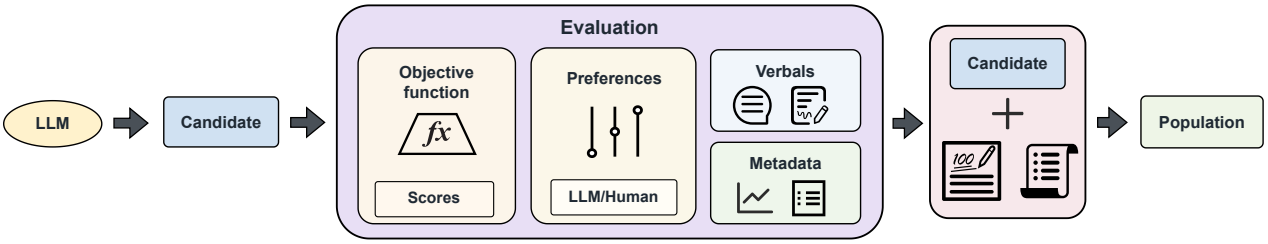


Figure 12: The structure of evaluation outputs in LLM-ES systems.

later scalar quantity is derived only after aggregating these relational signals.

DUEL-EVOLVE is the clearest pairwise-preference example: it does not start from a conventional numeric evaluator, but first collects pairwise preference judgments and then aggregates them into latent utilities with uncertainty estimates [66]. EvoCAD and Lark are the closest ranking-based examples: both first obtain evaluator-provided ordinal rankings and then transform those rankings into fitness-like signals for evolution [110, 126]. This design is suitable for settings where candidate quality is easier to compare than to quantify directly, such as open-ended generation tasks or problems with strong subjective or context-dependent criteria.

8.2. Additional Evaluation Feedback

This subsection focuses on *additional evaluation feedback beyond the objective value itself*. In these systems, evaluation does not end with returning one scalar score or one objective vector. Instead, it also produces extra feedback tied to the candidate, which can be preserved and reused in later search steps. Such feedback makes evaluation not only a selection signal, but also an information source for subsequent generation and revision.

8.2.1. Metadata

One common form of such additional feedback is *metadata*, namely auxiliary structured information returned together with the objective value during evaluation, such as correctness flags, exception types, profiler measurements, per-instance metrics, penalty codes, decomposed reward components, or other structured verification results. It does not replace the final score, but provides a more fine-grained description of how that score was obtained and what happened during execution, validation, or verification. Compared with a single summary number, such metadata can better support later diagnosis and targeted revision.

AlphaEvolve and KernelEvolve both retain richer evaluation outputs than one objective alone, including correctness-related status together with execution or profiling metadata [84, 105]. ThetaEvolve and LtE both expose shaped reward components or derived credit signals that explain why a candidate or trajectory received its final optimization signal [82, 140]. Strategic Search and Magellan preserve detailed timing or tuning information [10, 16]. [44] returns validation diagnostics, soundness counterexamples, and unsoundness costs rather than only a final pass/fail outcome. EVOSYN preserves structured verification artifacts such as executable pass/fail outcomes, consistency checks, and strategy scores derived from testing behaviors [31].

8.2.2. Verbalized Evaluation Feedback

Structured metadata is useful for recording what happened during evaluation, but it is often less suitable for directly guiding the next generation step. In many cases, the useful signal lies not in one isolated field, but in how multiple evaluation outcomes should be interpreted together. Natural-language feedback is therefore useful for summarizing such information into a more compact form and passing revision-relevant signals to the LLM more directly.

GEPA is the clearest example because its feedback function returns not only a numeric score but also free-form text and evaluation traces [2]. Similar behavior appears in AlphaApollo [171], PACEvolve [149], and REvolution [98], where evaluator outputs are summarized into comments, judgments, or traces rather than being left as raw logs only.

8.3. Evaluation Procedures

In many domains, direct full evaluation is too slow, too noisy, or too failure-prone to be used uniformly for every candidate. As a result, recent frameworks redesign the evaluation loop in several ways. Some organize evaluation into stages, some replace full evaluation with approximate signals, some accelerate evaluation through parallel execution, and some improve robustness through repeated measurement. Table 5 summarizes these representative evaluation procedures.

8.3.1. Staged Evaluation

Staged evaluation begins with lower-cost screening stages and only then promotes surviving candidates to more expensive tests. This helps reduce evaluation cost by filtering out weak or invalid candidates early. Existing systems mainly differ in what they use as the intermediate gate. AlphaResearch [156], Applied AlphaEvolve [6], and FOREAGENT [169] use early performance-oriented screening such as reward-model filtering, minibatches, validation splits before broader validation. AgentLTV [142], ArchAgent [46], and ThetaEvolve [140] instead use execution, verification, or constraint checks to decide whether downstream evaluation should continue.

Table 5
Representative evaluation procedures in LLM-ES systems.

Procedure	Mechanism	Characteristics	Papers
Staged evaluation	Multi-stage screening from cheaper tests to more expensive ones	Reduces full-evaluation cost by filtering weak candidates early; effectiveness depends on the quality of intermediate gates.	AlphaResearch [156]; ArchAgent [46]; ThetaEvolve [140]
Approximate evaluation	Proxy, partial, or reduced-fidelity evaluation during search	Improves efficiency when exact evaluation is too costly; may introduce bias relative to the final objective.	[42]; MadEvolve [80]
Parallel evaluation	Concurrent evaluation across multiple workers, devices, or environments	Improves throughput for expensive evaluation workloads; requires additional infrastructure and scheduling.	CodeEvolve [4]; DeltaEvolve [63]; KernelEvolve [84]
Repeated evaluation	Multiple runs with aggregated results	Improves robustness under noisy or workload-dependent measurements; increases per-candidate evaluation budget.	cuPilot [18]; EvolveSignal [136]; PromptBridge [139]

8.3.2. Approximate Evaluation

Approximate evaluation addresses the same full-evaluation bottleneck from another angle: instead of deciding which candidates should receive complete evaluation, it replaces exact full evaluation itself with approximate, proxy, or partial signals during search. Mathematical exploration and discovery at scale is a strong example: it explicitly uses multiple verifiers with different speed-accuracy tradeoffs and employs approximate geometric scoring procedures when exact evaluation would be too costly [42]. MadEvolve follows the same direction by using reduced-fidelity evaluators during search, such as lower-resolution settings or limited simulation setups, to avoid the cost of stronger full evaluation at every step [80].

8.3.3. Parallel Evaluation

Parallel evaluation addresses the same evaluation-speed bottleneck when the evaluation process itself can be parallelized. In such cases, candidate assessments are distributed across multiple workers, devices, or execution environments rather than being performed serially. CodeEvolve [4], DeltaEvolve [63], and ArchAgent [46] all employ parallel or distributed evaluation to keep the search loop tractable under expensive execution workloads. KernelEvolve follows the same direction with asynchronous remote dispatch to hardware pools [84].

8.3.4. Repeated Evaluation

Repeated evaluation addresses a different issue from the previous designs. Rather than reducing evaluation cost directly, it is used when evaluation outcomes are noisy, unstable, or workload-dependent, so that repeated runs can provide a more reliable estimate. cuPilot and EvolveSignal explicitly average over repeated timed runs or multiple scenarios to stabilize noisy performance measurements [18, 136]. PromptBridge and Making Databases Faster with LLM Evolutionary Sampling use the same basic mechanism more generally, where repeated trials and aggregated statistics are built into evaluation itself rather than treated as an afterthought [38, 139].

9. Exploration-Exploitation Control

As a metaheuristic search paradigm, LLM-guided evolutionary search faces the same exploration–exploitation challenge as classical evolutionary computation. Compared with classical EC, however, exploration and exploitation in LLM-ES depend simultaneously on multiple coupled factors, including parent selection, generation-operator selection, and foundation-model selection, which makes the design space both broader and more complex. At a high level, exploitation means continuing to generate better descendants along the search process associated with the currently strongest solutions, whereas exploration means shifting toward other search directions to escape local convergence. Some of these issues have already appeared in previous sections through archive and parent-selection design, generation mechanism selection, and model selection. This section therefore moves to a more global perspective and discusses how LLM-ES systems control the overall balance between exploration and exploitation. Table 6 summarizes the main explore–exploit control strategies discussed in this section.

9.1. Implicit Exploration-Exploitation Strategies

In many explicit explore–exploit designs, the two behaviors are realized through different contexts or control signals so that the LLM is guided to generate different kinds of responses. By contrast, some LLM-ES systems do not explicitly distinguish between exploratory and exploitative generation processes. Instead, they reuse one unified generation process and rely on the inherent stochasticity of LLM generation, together with differences in sampling, memory reuse, or archive exposure, to produce outcomes that behave differently in exploration and exploitation.

AlphaEvolve is the clearest example: it samples parents and inspirations from a persistent program database, proposes structured edits, evaluates offspring, and reinserts promising solutions [105]. Exploration arises from stochastic generation, archive diversity, and the resurfacing of multiple prior ideas, while exploitation arises from repeatedly refining strong stored programs. ThetaEvolve [140] and [73] follow the same broad logic, although with stronger archive

Table 6
Summary of exploration-exploitation control strategies in LLM-guided evolutionary systems.

Strategy class	Core mechanism	Characteristics	Papers
Implicit strategies	Exploration and exploitation emerge indirectly from archive structure, parent sampling, repeated proposals, and retention	Strong systems may balance search without an explicit mode flag; control is distributed across memory, ranking, and stochastic generation rather than concentrated in one controller	AlphaEvolve [105]; ThetaEvolve [140]; EvoSLD [85]
State-independent control	Use fixed rules or predefined stage progressions to determine how exploration and exploitation are allocated over the run	Simple, reproducible, and easy to interpret; makes the search schedule explicit, but the allocation is predetermined before the run begins	RankEvolve [103]; MadEvolve [80]; AgentLTV [142]
Adaptive control	Online controller adjusts search pressure from progress, uncertainty, tree statistics, or bandit-style reward signals	Treats explore–exploit balance as a feedback-control or resource-allocation problem; more responsive, but adds controller design and tuning complexity	AdaEvolve [11]; AgentLTV [142]; SEA-TS [148]; AdaExplore [34]; TIDE [15]; ShinkaEvolve [74]

boundaries than AlphaEvolve. In these systems, there is no dedicated switching variable that says "explore now" or "exploit now." Instead, the balance is realized through ranked database sampling, multiple rollouts, archive truncation, or persistent state reuse. EvoSLD also fits this pattern: it samples high-scoring parent subroutine pairs from a database, exposes additional high-performing pairs as contextual references, and relies on LLM mutation to generate diverse descendants without introducing a separate exploration-exploitation controller [85].

9.2. State-Independent Exploration-Exploitation Control

When exploration and exploitation are explicitly realized through different generation processes, the system gains a clearer way to separate broad search from local refinement. In this setting, however, a new question immediately arises: when should the current evolutionary step emphasize exploration, and when should it emphasize exploitation? One broad answer is to decide this independently of the current search state. In such systems, the control rule is specified in advance rather than adjusted online from observed progress, uncertainty, or stagnation.

The simplest state-independent designs use fixed rules throughout the search process. MadEvolve alternates between full rewrites and differential patches under a preset allocation [80], while RankEvolve chooses among exploratory, exploitative, and intermediate parent-sampling modes under a predefined distribution [103]. Other systems use predefined stage schedules, where the search pressure changes over time according to a hand-designed progression across stages. [137] and [135] both place broader exploration earlier and more conservative exploitation later. AgentLTV [142] and [116] follow the same exploration-first, exploitation-later logic, but there the shift acts directly on the outer search mode itself.

The main advantage of such state-independent control is interpretability. A user can identify in advance how exploratory and exploitative behavior will be allocated. This simplicity makes these systems comparatively easy to reproduce and analyze, although the resulting search pressure may not match the realized trajectory of every run.

9.3. Adaptive Exploration-Exploitation Strategies

The limitation of state-independent control is that the explore–exploit balance is predetermined before the run begins. In practice, however, the search process can stagnate earlier than expected, continue benefiting from exploration longer than expected, or require stronger exploitation in only a subset of branches. This creates the need for adaptive strategies, which adjust the explore–exploit balance online according to the observed search state. The main benefit of such designs is that they can respond more flexibly to actual progress, uncertainty, or resource usage during evolution.

AdaEvolve adjusts each island's exploration intensity from recent improvements, so stagnation increases exploratory sampling while strong progress increases exploitative refinement [11]. [10] uses a strategist conditioned on the current search context to choose whether to *innovate*, *combine*, or *refine*. AgentLTV [142], SEA-TS [148], and AdaExplore [34] instead adapt search pressure through tree statistics, using PUCT-, UCT-, or MCTS-style selection to trade off exploration against refinement during the run. TIDE [15], ShinkaEvolve [74], and SMCEvolve [64] apply the same general idea to prompt strategies, LLM choices, island-level resource allocation, or annealing-based search-pressure control. CausalEvolve gives a related factor-level variant: it uses a bandit-style controller to alternate between exploratory random interventions and exploitative best-action reuse [21]. Taken together, these systems treat explore–exploit balance as an online control problem that is continuously updated from observed feedback during search.

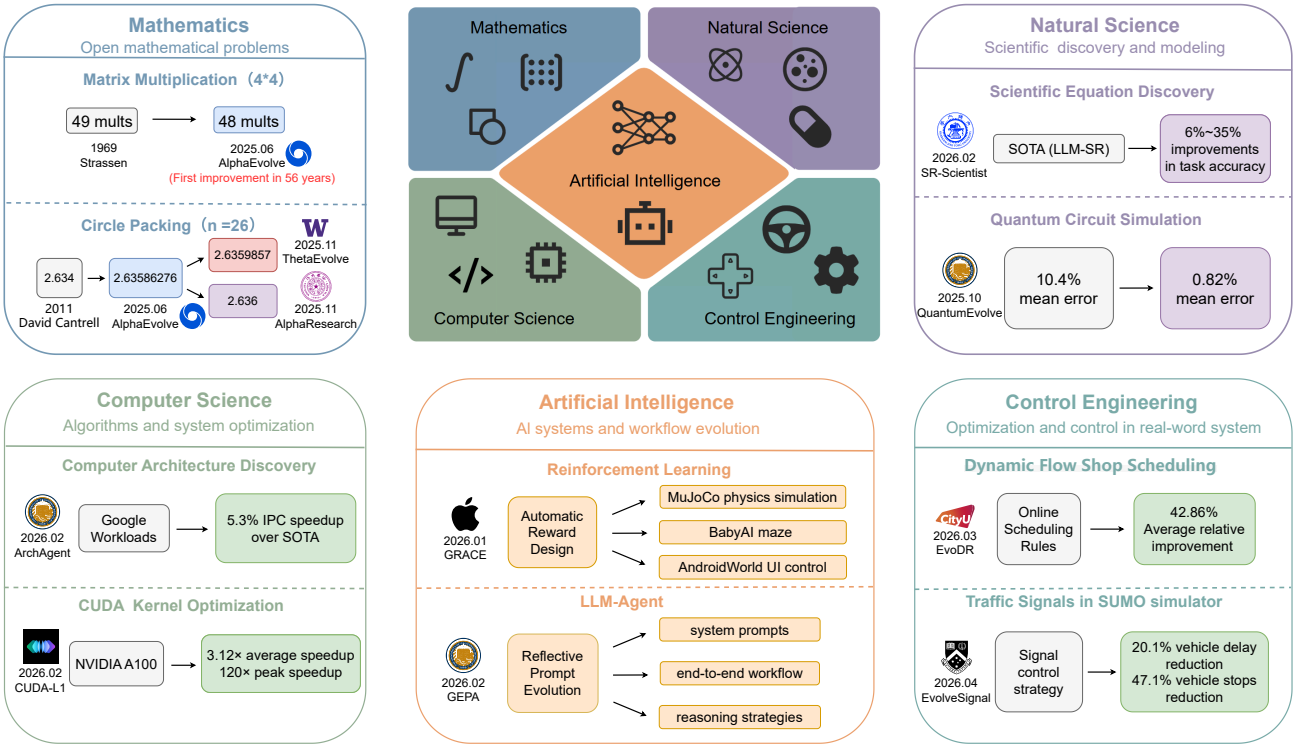


Figure 13: Major application domains of LLM-ES: Representative tasks and representative breakthroughs.

10. Applications and Open-Source Frameworks

By combining semantic generation with iterative feedback-driven search, LLM-ES has achieved applications and representative results across a wide range of domains. It is especially suitable for complex structured optimization problems whose solutions are difficult to hand-design but can still be evaluated through executable, simulatable, or verifiable feedback. In some cases, the discovered solutions even surpass the best human-designed results. Figure 13 summarizes the five core application domains of LLM-ES, presenting representative tasks and their corresponding state-of-the-art breakthroughs with quantitative performance improvements. These results collectively demonstrate the paradigm's broad applicability and superior performance across diverse scientific and engineering fields.

10.1. Applications

10.1.1. Mathematical Sciences

Mathematical-science problems often have highly structured search spaces, strong symbolic constraints, and clear verification procedures, but they also tend to suffer from severe combinatorial explosion and heavy reliance on expert-designed heuristics. This makes them especially suitable for LLM-ES. Existing work has already demonstrated representative progress in several major directions.

Open-Problem Exploration Open mathematical problems lie at the core of pure mathematics and typically lack clear solution pathways, making systematic breakthroughs challenging for traditional approaches.

AlphaEvolve provides the strongest early evidence that LLM-ES can make nontrivial progress on open mathematical problems [105]. Across open tasks spanning analysis, combinatorics, number theory, and geometry, it matches the best known constructions in 38 cases and surpasses them in 14 cases. Its most striking result is a new 4×4 matrix multiplication algorithm, which constitutes the first improvement in this setting after 56 years of work. ThetaEvolve shows that such progress is not limited to large closed-source model ensembles [140]: using a small open-source model, it obtains new best-known bounds on open problems highlighted by AlphaEvolve. AlphaResearch further demonstrates that LLM-ES can be competitive with human researchers in open-ended mathematical algorithm discovery [156]. On its AlphaResearchComp benchmark of eight open-ended problems, it achieves a 2/8 win rate in head-to-head comparison against human researchers.

Heuristic Discovery Heuristics denote manually designed algorithmic rules, operators, or decision policies used to construct or improve approximate solutions for computationally intractable optimization problems. Such heuristics are central to many hard discrete optimization tasks, but their manual design is often time-consuming and highly dependent on domain expertise.

TIDE demonstrates strong heuristic-discovery performance of LLM-ES across multiple NP-hard combinatorial

optimization problems [15]. Across nine combinatorial optimization tasks and five algorithmic frameworks, it consistently achieves the best results among LLM-based automated heuristic design methods. For SAT solving, AutoModSAT reports more than 50% performance improvement over its baseline modular solver and average gains exceeding 30% over existing state-of-the-art solvers [125]. For multi-objective combinatorial optimization, MPaGE consistently outperforms prior LLM-based heuristic-design methods across multiple benchmarks [52]. Related mathematical applications in this direction include adversarial heuristic generation for combinatorial optimization [104], game-theoretic co-evolution for heuristic discovery [68], ranking-function evolution for canonical blow-up construction [9], evaluation of LLM-evolved 2D bin-packing heuristics [59], and reinforced generation of combinatorial structures for complexity-theoretic constructions [101].

Knowledge Discovery Automated Knowledge discovery aims to enable intelligent systems to uncover mathematical knowledge across different levels of abstraction, ranging from data-driven equation identification to high-level theory formation. This direction serves as a paradigm bridging empirical observations and formal mathematical reasoning.

In symbolic regression, IdeaSearchFitter demonstrates that LLM-ES can recover mathematical expressions and physical laws directly from data with high accuracy and robustness [121]. On the FSReD benchmark, it outperforms several strong baselines in both noise-free and noisy settings, while also producing compact and interpretable expressions on real-world datasets. At a higher level of abstraction, EvoAbstract shows that LLM-ES can support automated mathematical theory formation rather than only equation fitting [128]. In the FERMAT environment for elementary number theory and finite fields, its evolved interestingness measures outperform manually designed baselines and rediscover core concepts such as powers, primality, and finite-field constructions.

10.1.2. Computer Science

LLM-ES systems have been successfully applied to automated optimization across the full stack of computer systems, spanning hardware design, system scheduling, compiler optimization, and kernel generation.

At the system level, AlphaEvolve has been applied to multiple components of Google's computing infrastructure, including data center scheduling, large model training, hardware circuit design, and low-level kernel optimization, consistently improving system efficiency and reducing optimization time [105].

In computer architecture, ArchAgent automates the design of last-level cache replacement policies [46]. By leveraging cycle-accurate simulation, it generates optimized strategies for both standard benchmarks and real-world workloads, while supporting workload-aware runtime tuning to improve processor performance.

At the compiler and kernel level, this paradigm enables the automatic discovery of optimization strategies traditionally crafted by experts. Magellan synthesizes compiler heuristics for tasks such as function inlining, register allocation, and computation graph transformation, producing deployable optimization rules that improve performance across compiler frameworks [16]. KernelEvolve further extends this idea to heterogeneous AI systems, where it automatically generates and optimizes high-performance kernels for deep learning workloads across diverse hardware platforms [84]. The same paradigm has also been applied to Gaussian splatting kernels, showing that LLM-guided code evolution can optimize graphics-oriented low-level kernels beyond standard deep-learning operators [57].

10.1.3. Artificial Intelligence

Artificial intelligence presents a broad class of design problems in which the target object is related to neural networks, reward functions, agent pipelines, etc. These objects are typically difficult to optimize manually, while their quality can often be assessed through task performance or downstream system feedback.

At a more general machine-learning level, evolutionary methods have been used to discover scaling laws that characterize how model performance changes with training and model size [85]. The same paradigm also supports programmatic representation learning [109]. In neural rendering and representation design, EvoRender applies LLM-guided evolutionary search to neural texture compression decoder discovery and reports architectures that outperform hand-designed baselines [97].

For LLM agents and reasoning systems, evolutionary optimization has been used to refine prompts, tool usage, execution configurations, and reasoning trajectories. [8] and GEPA [2] show that such optimization can consistently improve reasoning and code-generation performance without parameter updates. [132] further strengthens deep reasoning through iterative refinement over candidate reasoning paths, while [166] improve the accuracy, stability, and efficiency of mathematical reasoning across models of different scales. In advanced text generation, WRitEer [155] combines multi-agent collaboration, evolutionary prompt refinement, and preference-based fine-tuning to improve human-like writing quality.

In reinforcement learning, [72] automatically evolves interpretable and verifiable environment dynamics for model-based reinforcement learning, GRACE [118] searches over executable reward functions for inverse reinforcement learning, and [83] search discovers multi-agent learning algorithms for imperfect-information games.

LLM-ES has also been applied to AI systems and infrastructure optimization. ASAP [30] uses a multi-agent framework to diagnose and improve distributed LLM training bottlenecks across computation, memory, and communication, while ContextEvolve [123] combines multi-agent context compression with evolutionary search to optimize system-level code under limited context budgets.

10.1.4. Natural Sciences

Natural-science problems focus on discovering governing laws, mechanistic models, or domain-specific computational procedures from observational data, simulations, and experiments.

In scientific equation discovery, SR-SCIENTIST searches for interpretable equations from data across physics, chemistry, biology, and materials science, outperforming baselines by absolute margins of 6%–35% [144]. In quantum simulation, evolutionary program synthesis optimizes executable circuit-generation code for molecular structure learning and many-body dynamics, reducing approximation error from 10.4% to 0.82% [162]. In cognitive and behavioral science, DataDIVER discovers interpretable computational models of human and animal learning directly from behavioral data and uses them to surface new mechanistic insights [67]. In computational biology, VCHarness applies autonomous program search to perturbation-response model construction for virtual cells, outperforming expert-designed baselines [24]. In ecological monitoring, the bioacoustic-association framework addresses the constrained problem of multi-channel whale-call association, discovering algorithms that generalize to unseen synthetic regimes and transfer effectively to real-world whale datasets [99]. In molecular and drug discovery, MCCE tackles five-objective molecule design and reports state-of-the-art Pareto-front quality [113].

10.1.5. Control and Engineering Systems

Control and engineering systems typically require executable policies or rules that operate under dynamics and uncertainty. These problems are well suited to LLM-ES because candidate solutions can be evaluated directly in simulators or operational environments, allowing iterative search over structured control logic rather than only parameter tuning.

In intelligent manufacturing, this paradigm automatically generates and refines dispatching rules for dynamic flexible job-shop scheduling under stochastic and multi-stage settings, and has also been extended to dynamic hybrid flow shop scheduling through personalized multi-island reflective evolution [76, 112]. In power system scheduling, theory-driven modeling and LLM-guided evolution are used to discover warm-start scoring functions and hyperparameter configurations for large-scale unit commitment and economic dispatch problems [164]. In transportation and logistics, VRPAgent targets multiple vehicle-routing problems, and reports heuristic operators that outperform prior state-of-the-art VRP heuristics [55]. In traffic control, it evolves signal timing policies in simulators such as SUMO, yielding lightweight and effective strategies that outperform classical heuristics while remaining practical for real-world deployment [136]. Related work also studies robot path planning by combining it with reinforcement learning, treating path-planning control policies as executable objects that can be evaluated and improved in closed loop [45].

10.1.6. Emerging Domains

Beyond the major domains above, recent LLM-ES applications have also begun to target newer areas. In quantitative finance, EvoRisk autonomously discovers a regime-adaptive risk metric for asset selection and portfolio optimization under volatility shifts, jump risk, and persistent drawdowns [158]. In biomedical imaging, EvoAgents applies LLM-guided evolutionary search to automatically generate and refine convolutional architectures for cell nuclei detection from microscopy images [3]. In social simulation and AI evaluation, Persona Generators evolves reusable code that expands compact context descriptions into diverse synthetic populations for stress-testing human-facing AI systems across a wider range of possible user behaviors [108].

10.2. Open-Source Frameworks

As LLM-ES has expanded across application domains, a corresponding resource ecosystem has also emerged. Open-source implementations expose reusable search loops, evaluators, and engineering pipelines that support later reproduction and extension. Table 7 summarizes representative open-source frameworks.

11. Benchmarks

As LLM-ES has expanded across tasks and domains, a corresponding benchmark ecosystem has also emerged. Some of the benchmarks listed in Table 8, from ADRS [23] to MLEBench [13], were not originally introduced specifically for LLM-ES, but have been repeatedly adopted as public evaluation targets in later work. By contrast, more recent benchmarks such as FrontierCS [96] and FML-bench [172] were introduced with concerns that are more directly tied to this area. Their papers repeatedly argue that many existing evaluations are built around known answers, pass-or-fail grading, isolated code snippets, or end metrics that reveal little about iterative search and research behavior.

The motivations given in these newer benchmark papers are also fairly consistent. FrontierCS [96] emphasizes open-ended computer-science problems whose optima are unknown but whose solutions remain executable and deterministically scorable. HeuriGym [17] justifies combinatorial optimization as a setting with clear objectives and large solution spaces for iterative heuristic improvement. AIRS-Bench [93] argues for a standardized framework to compare AI research agents across full research-lifecycle tasks. FML-bench [172] emphasizes fundamental machine-learning research problems and metrics that capture iterative research behavior rather than only final performance and cost. CktEvo [119] highlights repository-level RTL evolution, where evaluation must account for cross-file dependencies and toolchain-in-the-loop feedback rather than isolated modules. Taken together, these benchmarks indicate that evaluation in this area is moving toward settings that are more open-ended, executable, and process-aware, while still remaining objectively checkable. Representative publicly available benchmarks are summarized in Table 8.

Table 7
Representative open-source frameworks in LLM-ES literature.

Framework	Tested scenario	Representative contribution	Highlights
GEPA [2][1]	Prompt optimization	natural-language reflection; Pareto-based candidate selection	prompt-centric design; code-optimization transfer
ShinkaEvolve [74][2]	Open mathematical optimization; algorithm design; machine-learning pipeline optimization	adaptive parent sampling; novelty rejection sampling; bandit-based LLM ensemble selection	interactive visualization; sample-efficient search
CodeEvolve [4][3]	Open mathematical optimization; algorithm design	island-based search; modular LLM orchestration	transparency; ablation support
GigaEvo [69][4]	Open mathematical optimization; algorithm design	DAG-based evaluation pipelines; MAP-Elites evolution; lineage-aware mutation context	declarative configuration; rapid experimentation
ThetaEvolve [140][5]	Open mathematical optimization	test-time reinforcement learning; batched program evolution	small-model setting; continual test-time learning
LoongFlow [133][6]	Open mathematical optimization; machine-learning pipeline optimization	Plan-Execute-Summarize prompting; hybrid evolutionary memory	pre-built agents; reasoning-guided search
AdaptEvolve [114][7]	Algorithm design	adaptive multi-LLM routing; confidence-driven model selection	cost efficiency
OR-Agent [89][8]	Algorithm design; cooperative driving	tree-structured exploration; evolutionary-systematic ideation; hierarchical reflection	workflow-oriented design; structured research management
SkyDiscover [91][9]	Open mathematical optimization; systems optimization; algorithm design	adaptive search intensity control; resource allocation; search-strategy evolution	shared AdaEvolve/EvoX repository; live monitoring; human-in-the-loop steering

1. https://github.com/gepa-ai/gepa 2. https://github.com/SakanaAI/ShinkaEvolve 3. https://github.com/inter-co/science-codeevolve
4. https://github.com/AIRI-Institute/gigaevo-core 5. https://github.com/ypwang61/ThetaEvolve 6. https://github.com/baidu-baige/LoongFlow
7. https://github.com/raypretam/adaptive_llm_selection 8. https://github.com/qiliuchn/OR-Agent 9. https://github.com/skydiscover-ai/skydiscover

12. Conclusion

12.1. Summary of Key Findings

LLM-guided evolutionary search can be understood as an important form of LLM test-time scaling. Compared with repeated sampling or simply extending one chain of thought, LLM-ES improves performance more systematically by increasing generation attempts, using search state to steer later generations, and maintaining candidates whose evaluated quality keeps improving under the target objective. Because of this potential, the area has developed rapidly, but the relationships among existing systems have remained insufficiently organized. This survey attempts to classify existing works from the main layers of an LLM-ES pipeline, including search problem formulation, population design, parent selection, candidate generation mechanisms, prompt builder design, foundation-model use, candidate evaluation, exploration–exploitation control, and the surrounding application and resource ecosystem. The goal is to provide a quick guide for new researchers and a clearer systems-level perspective for more experienced audience.

12.2. Open Challenges and Future Directions

12.2.1. Aligning Foundation Models with Evolutionary Search

One major direction is to make the foundation model itself better aligned with evolutionary search. Most current LLM-ES systems keep the backbone largely fixed and focus mainly on designing the search loop above it. This has been effective, but it also means that the model is usually not optimized for iterative revision, long-horizon search, repeated evaluator-conditioned improvement, or coordinated multi-role interaction. A natural next step is therefore to adapt the model directly from search trajectories, successful revisions, and verifier feedback, as early systems with in-loop model updating already begin to suggest [113, 114]. An initial attempt in this direction adapts the model from search-generated algorithm databases rather than leaving the backbone entirely unchanged [88]. More broadly, closer integration with multi-agent reinforcement-learning and reasoning-oriented training paradigms may help the model become not only a better generator of candidates, but also a better search participant that can revise, coordinate, and reason more effectively across many evolutionary iterations.

Table 8
Representative publicly available benchmarks related to LLM-ES.

Benchmark	Domain / Task	Representative use	Scale
ADRS [23][1]	Systems optimization	Used by [11, 123]	7 tasks
ARC-AGI-2 [26, 27][2]	Abstract reasoning and program induction	Used by [11, 132]	120 tasks
LiveCodeBench [62][3]	Code generation	Used by [31, 114, 132]	400 tasks
BBOB / CoCo-BBOB [49][4]	Black-box optimization	Used by [1, 134]	2160 tasks
ALE-Bench [60][5]	Long-horizon algorithm engineering	Used by [8, 74, 78]	40 tasks
GSM8K [28][6]	Math reasoning	Used by [8, 166]	1319 tasks
KernelBench [106][7]	GPU kernel code optimization	Used by [78, 84, 100, 149]	250 tasks
MLEBench [13][8]	Machine-learning engineering and competitions	Used by [43, 78, 133]	75 tasks
FrontierCS [96][9]	Open-ended computer-science coding and algorithm design	Tests models such as GPT-5, Gemini, Claude, Grok, and DeepSeek	156 tasks
FML-bench [172][10]	Automatic machine-learning research on real codebases	Tests [65, 92] and Claude Code	8 tasks
AIRS-Bench [93][11]	Frontier AI research science agents	Tests one-shot, Greedy, and [152]	20 tasks
HeuriGym [17][12]	Heuristic design for combinatorial optimization	Tests direct LLM baselines and evolutionary frameworks such as [29, 87, 153]	9 tasks
CktEvo [119][13]	Repository-level RTL design evolution	Tests its own reference closed-loop RTL evolution agent	11 tasks

1. https://github.com/UCB-ADRS/ADRS 2. https://github.com/arcprize/ARC-AGI-2 3. https://github.com/LiveCodeBench/LiveCodeBench
4. https://github.com/numbbo/coco 5. https://github.com/SakanaAI/ALE-Bench 6. https://huggingface.co/datasets/openai/gsm8k
7. https://github.com/ScalingIntelligence/KernelBench 8. https://github.com/openai/mle-bench 9. https://www.frontier-cs.org/
10. https://github.com/qrzou/FML-bench 11. https://github.com/facebookresearch/airs-bench 12. https://github.com/cornell-zhang/heurigym
13. https://github.com/cure-lab/cktevo

12.2.2. Moving Beyond Near-Zeroth-Order Optimization

Another challenge is to move beyond the current reliance on near-zeroth-order optimization [11]. Most existing methods still depend mainly on candidate generation, trial-and-error evaluation, and selection, while making limited use of higher-order information about how the search landscape is changing. This limitation matters because pure black-box iteration can be expensive and slow, especially in long-horizon problems with costly evaluation. A promising direction is to incorporate more structured update signals, surrogate local models, or optimization-inspired state that can accelerate search beyond simple sampling and filtering. Recent work such as TextBFGS points to this possibility by explicitly modeling a text-space analogue of BFGS-style updates [167], but much broader exploration is still needed.

12.2.3. Building Human-in-the-Loop Evolutionary Workflows

A further challenge is to build more effective human-in-the-loop evolutionary workflows. Current systems are dominated by fully automated loops, while human guidance often enters only through task formulation, benchmark choice, or occasional manual intervention outside the formal search process. In many realistic research and engineering settings, however, humans are important not only because they can provide preferences or corrections, but also because they can reformulate objectives, inject domain knowledge, and redirect search when automatic evaluation is incomplete. Recent systems already hint at this direction by mixing LLM search with explicit human simplification or review steps [104], but making such collaboration more systematic remains an open problem.

12.2.4. Symbolic Theorem Proving

Mathematical discovery and theorem proving form another important frontier. Symbolic regression and theory-formation systems already show that LLM-ES can contribute to mathematical pattern discovery, but stronger integration with formal proving, verifier-guided search, and theorem-oriented reasoning may substantially expand this line of work. Even though the technical route is different from typical LLM-ES pipelines, recent progress in Olympiad-level formal mathematical reasoning, such as AlphaProof, suggests the broader promise of combining search, verification, and learning for difficult mathematical problems [58].

12.2.5. Expanding Toward High-Value Real-World Applications

LLM-ES is likely to expand further toward real-world directions with clearer commercial or scientific value. Much of the current literature still concentrates on algorithm discovery, synthetic benchmarks, or well-scoped scientific

tasks. Going forward, more attention may shift toward deployment-relevant settings such as auto bidding, safety-constrained industrial optimization, and high-impact scientific systems including controlled nuclear fusion. These domains place stricter demands on reliability, efficiency, and interpretability, but they are also where stronger LLM-ES systems may ultimately deliver the most practical value.

References

[1] Achtelik, W., Guzowski, H., Smołka, M., Mańdziuk, J., 2026. Automatic design of optimization test problems with large language models. arXiv:2602.02724.

[2] Agrawal, L.A., Tan, S., Soylu, D., Ziems, N., Khare, R., Opsahl-Ong, K., Singhvi, A., Shandilya, H., Ryan, M.J., Jiang, M., Potts, C., Sen, K., Dimakis, A.G., Stoica, I., Klein, D., Zaharia, M., Khattab, O., 2026. Gepa: Reflective prompt evolution can outperform reinforcement learning. arXiv:2507.19457.

[3] Alonso, S., Bauer, A., 2025. Evoagents: Generative ai for rapid architecture search in cell nuclei detection, in: IEEE International Conference on eScience (eScience), IEEE. pp. 341–342.

[4] Assumpção, H., Ferreira, D., Campos, L., Murai, F., 2026. Codeevolve: an open source evolutionary coding agent for algorithmic discovery and optimization. arXiv:2510.14150.

[5] Aygün, E., Belyaeva, A., Comanici, G., Coram, M., Cui, H., Garrison, J., Kast, R.J.A., McLean, C.Y., Norgaard, P., Shamsi, Z., Smalling, D., Thompson, J., Venugopalan, S., Williams, B.P., He, C., Martinson, S., Plomecka, M., Wei, L., Zhou, Y., Zhu, Q.Z., Abraham, M., Brand, E., Bulanova, A., Cardille, J.A., Co, C., Ellsworth, S., Joseph, G., Kane, M., Krueger, R., Kartiwa, J., Liebling, D., Lueckmann, J.M., Raccuglia, P., Xuefei, Wang, Chou, K., Manyika, J., Matias, Y., Platt, J.C., Dorfman, L., Mourad, S., Brenner, M.P., 2026. An ai system to help scientists write expert-level empirical software. arXiv:2509.06503.

[6] Beresnev, D., Khalikov, R., Tolmachev, A., Ulitin, I., Zakirova, A., 2025. Applied alphaevolve .

[7] Bouscary, M., Wu, M., Amin, S., 2026. Budget-efficient automatic algorithm design via code graph. URL: https://arxiv.org/abs/2605.10598, doi:10.48550/arXiv.2605.10598, arXiv:2605.10598.

[8] Brookes, P., Voskanyan, V., Giavrimis, R., Truscott, M., Ilieva, M., Pavlou, C., Staicu, A., Adham, M., Evers-Hood, W., Gong, J., Zhang, K., Fedoseev, M., Sharma, V., Bauer, R., Wang, Z., Nair, H., Jie, W., Xu, T., Constantin, A., Kanthan, L., Basios, M., 2025. Evolving excellence: Automated optimization of llm-based agents. URL: https://arxiv.org/abs/2512.09108, arXiv:2512.09108.

[9] Bérczi, G., 2026. Evolving ranking functions for canonical blow-ups in positive characteristic. arXiv:2602.06553.

[10] Carreon, A., Sharma, V., Raman, V., 2025. Automated design optimization via strategic search with large language models. arXiv:2511.22651.

[11] Cemri, M., Agrawal, S., Gupta, A., Liu, S., Cheng, A., Mang, Q., Naren, A., Erdogan, L.E., Sen, K., Zaharia, M., Dimakis, A., Stoica, I., 2026. Adaevolve: Adaptive llm driven zeroth-order optimization. arXiv:2602.20133.

[12] Cen, S., Tan, Y., 2025. Beyond algorithm evolution: An llm-driven framework for the co-evolution of swarm intelligence optimization algorithms and prompts. arXiv:2512.09209.

[13] Chan, J.S., Chowdhury, N., Jaffe, O., Aung, J., Sherburn, D., Mays, E., Starace, G., Liu, K., Maksin, L., Patwardhan, T., et al., 2025. Mle-bench: Evaluating machine learning agents on machine learning engineering, in: International Conference on Learning Representations, pp. 50466–50494.

[14] Chauhan, D., Dutta, B., Bala, I., van Stein, N., Bäck, T., Yadav, A., 2025. Evolutionary computation and large language models: A survey of methods, synergies, and applications. URL: https://arxiv.org/abs/2505.15741, doi:10.48550/arXiv.2505.15741, arXiv:2505.15741.

[15] Chen, C., Zhong, M., Fan, Y., Shi, J., Sun, J., 2026a. Tide: Tuning-integrated dynamic evolution for llm-based automated heuristic design. arXiv:2601.21239.

[16] Chen, H., Novikov, A., Vũ, N., Alam, H., Zhang, Z., Grossman, A., Trofin, M., Yazdanbakhsh, A., 2026b. Magellan: Autonomous discovery of novel compiler optimization heuristics with alphaevolve. arXiv:2601.21096.

[17] Chen, H., Wang, Y., Cai, Y., Hu, H., Li, J., Huang, S., Deng, C., Liang, R., Kong, S., Ren, H., Samaranayake, S., Gomes, C.P., Zhang, Z., 2026c. Heurigym: An agentic benchmark for llm-crafted heuristics in combinatorial optimization. arXiv:2506.07972.

[18] Chen, J., Wu, Q., Li, B., Ma, L., Si, X., Hu, Y., Yin, S., Yang, J., 2025a. cupilot: A strategy-coordinated multi-agent framework for cuda kernel evolution. arXiv:2512.16465.

[19] Chen, Y., Cui, W., Chen, Y., Tan, M., Zhang, X., Liu, J., Li, H., Zhao, D., Wang, H., 2025b. Robogpt: An llm-based long-term decision-making embodied agent for instruction following tasks. IEEE Transactions on Cognitive and Developmental Systems 17, 1163–1174. doi:10.1109/TCDS.2025.3543364.

[20] Chen, Y., Li, H., Chen, Y., Zhao, D., 2025c. Leaffordnav: Enhancing open-vocabulary mobile manipulation with llm-guided exploration and affordance-aware navigation, in: IEEE International Conference on Multimedia and Expo (ICME), pp. 1–6. doi:10.1109/ICME59968.2025.11208899.

[21] Chen, Y., Liu, C., Chen, Z., Liu, T., Han, B., Zhang, K., 2026d. Causalevolve: Towards open-ended discovery with causal scratchpad. arXiv:2603.14575.

[22] Chen, Y., Tian, S., Liu, S., Zhou, Y., Li, H., Zhao, D., 2025d. Conrft: A reinforced fine-tuning method for vla models via consistency policy, in: Proceedings of Robotics: Science and Systems, RSS. doi:10.15607/RSS.2025.XXI.019.

[23] Cheng, A., Liu, S., Pan, M., Li, Z., Wang, B., Krentsel, A., Xia, T., Cemri, M., Park, J., Yang, S., Chen, J., Agrawal, L., Desai, A., Xing, J., Sen, K., Zaharia, M., Stoica, I., 2025. Barbarians at the gate: How ai is upending systems research. arXiv:2510.06189.

[24] Cheng, X., Li, P., Guo, H., Liang, Y., Gong, J., de Vazelhes, W., Gou, C., Xie, P., Song, L., Xing, E., 2026. Harnessing ai to build virtual cells. bioRxiv , 2026–04.

[25] Chirra, S.R., Teoh, J., Paruchuri, P., Varakantham, P., 2026. On discovering algorithms for adversarial imitation learning, in: The Fourteenth International Conference on Learning Representations.

[26] Chollet, F., 2019. On the measure of intelligence. doi:10.48550/arXiv.1911.01547, arXiv:1911.01547.

[27] Chollet, F., Knoop, M., Kamradt, G., Landers, B., Pinkard, H., 2026. Arc-agi-2: A new challenge for frontier ai reasoning systems. doi:10.48550/arXiv.2505.11831, arXiv:2505.11831.

[28] Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., Schulman, J., 2021. Training verifiers to solve math word problems. doi:10.48550/arXiv.2110.14168, arXiv:2110.14168.

[29] Dat, P.V.T., Doan, L., Binh, H.T.T., 2025. Hsevo: Elevating automatic heuristic design with diversity-driven harmony search and genetic algorithm using llms, in: Proceedings of the AAAI Conference on Artificial Intelligence, pp. 26931–26938.

[30] Ding, Y., Chen, X., Zhang, X., Zhou, Z., 2025. ASAP: an agentic solution to auto-optimize performance of large-scale LLM training, in: Machine Learning for Systems.

[31] Du, H., Li, B., Yang, A., He, S., Guo, Q., Tao, D., 2025. Evosyn: Generalizable evolutionary data synthesis for verifiable learning. arXiv:2510.17928.

[32] Du, M., Yin, X., Luo, Y., Beniwal, D., Tang, S., Sharma, H., Cherukara, M.J., 2026a. Cvevolve: Autonomous algorithm discovery for unstructured scientific data processing. URL: https://arxiv.org/abs/2605.11359, doi:10.48550/arXiv.2605.11359, arXiv:2605.11359.

[33] Du, S., Yan, X., Shi, J., Cao, Z., Feng, S., Liang, Z., Sun, B., Peng, T., Zhou, Y., Li, X., Zhou, J., He, L., Zhang, B., Bai, L., 2026b. Mlevolve: A self-evolving framework for automated machine

learning algorithm discovery. URL: https://arxiv.org/abs/2606.06473, doi:10.48550/arXiv.2606.06473, arXiv:2606.06473.

[34] Du, W., Zhuo, J., Dong, Y., He, A.W., Sun, W., Zheng, Z., Karunaratne, M., Fox, I., Dettmers, T., Chen, T., Yang, Y., Welleck, S., 2026c. Adaexplore: Failure-driven adaptation and diversity-preserving search for efficient kernel generation. URL: https://arxiv.org/abs/2604.16625, doi:10.48550/arXiv.2604.16625, arXiv:2604.16625.

[35] Du, Y., Li, S., Torralba, A., Tenenbaum, J.B., Mordatch, I., 2023. Improving factuality and reasoning in language models through multiagent debate. doi:10.48550/arXiv.2305.14325, arXiv:2305.14325.

[36] Dwivedula, R., Saxena, D., Akella, A., Chaudhuri, S., Kim, D., 2025. Man-made heuristics are dead. long live code generators! arXiv:2510.08803.

[37] Eiben, A.E., Smith, J.E., 2015. Introduction to Evolutionary Computing. 2nd ed., Springer, Berlin, Heidelberg. doi:10.1007/978-3-662-44870-5.

[38] Erol, M.H., Hao, X., Bianchi, F., Greco, C., Tagliabue, J., Zou, J., 2026. Making databases faster with llm evolutionary sampling. arXiv:2602.10387.

[39] Fan, Y., Li, C., Xu, L., Pan, X., Dai, J., Geng, H., Yang, M., 2026. Cyberevolver: Structured self-evolution for cybersecurity agents on the fly. doi:10.48550/arXiv.2605.26195, arXiv:2605.26195.

[40] Fu, Y., Chen, T., Chai, J., Wang, X., Tu, S., Yin, G., Lin, W., Zhang, Q., Zhu, Y., Zhao, D., 2026. SRFT: A single-stage method with supervised and reinforcement fine-tuning for reasoning, in: The Fourteenth International Conference on Learning Representations.

[41] Garg, S., Singh, A., Nayak, G.K., 2026. Sidiffagent: Self-improving diffusion agent. arXiv:2602.02051.

[42] Georgiev, B., Gómez-Serrano, J., Tao, T., Wagner, A.Z., 2025. Mathematical exploration and discovery at scale. arXiv:2511.02864.

[43] Gideoni, Y., Risi, S., Gal, Y., 2026. Simple baselines are competitive with code evolution. arXiv:2602.16805.

[44] Gu, Q., Singh, A., Singh, G., 2025. Cost-driven synthesis of sound abstract interpreters. arXiv:2511.13663.

[45] Gu, S., 2025. Intelligent robot path planning performance optimization and control strategy integrating reinforcement learning and alphaevolve, in: 2nd International Conference on Electronic Circuits and Signaling Technologies (ICECST), IEEE. pp. 100–106.

[46] Gupta, R., Jain, A., Gonzalez, A., Novikov, A., Huang, P.S., Balog, M., Eisenberger, M., Shirobokov, S., Vũ, N., Dixon, M., Nikolić, B., Ranganathan, P., Karandikar, S., 2026. Archagent: Agentic ai-driven computer architecture discovery. arXiv:2602.22425.

[47] Han, X., Chen, S., Fu, Z., Feng, Z., Fan, L., An, D., Wang, C., Guo, L., Meng, W., Zhang, X., Xu, R., Xu, S., 2026. Multimodal fusion and vision-language models: A survey for robot vision. Information Fusion 126, 103652. doi:10.1016/j.inffus.2025.103652.

[48] Handa, D., Blincoe, D., Adams, O., Fu, Y., 2025. Optagent: Optimizing query rewriting for e-commerce via multi-agent simulation. arXiv:2510.03771.

[49] Hansen, N., Auger, A., Ros, R., Mersmann, O., Tušar, T., Brockhoff, D., 2021. Coco: A platform for comparing continuous optimizers in a black-box setting. Optimization Methods and Software 36, 114–144.

[50] Hansen, N., Ostermeier, A., 2001. Completely derandomized self-adaptation in evolution strategies. Evolutionary Computation 9, 159–195. doi:10.1162/106365601753382721.

[51] He, K., Mao, R., Lin, Q., Ruan, Y., Lan, X., Feng, M., Cambria, E., 2025. A survey of large language models for healthcare: from data, technology, and applications to accountability and ethics. Information Fusion 118, 102963. doi:10.1016/j.inffus.2025.102963.

[52] Hieu, H.M., Phan, H., Doan, T.D., Dao, T., Tran, C.D., Binh, H.T.T., 2026. Pareto-grid-guided large language models for fast and high-quality heuristics design in multi-objective combinatorial optimization. Proceedings of the AAAI Conference on Artificial Intelligence 40, 36964–36972. doi:10.1609/aaai.v40i43.41024.

[53] Ho, C.T., Gong, J., Yang, H., Akkur, A., Ren, H., 2026. Polymath: Self-improving hierarchical workflow for multi-domain problem solving, in: Proceedings of the International Symposium on Physical Design, pp. 121–130.

[54] Holland, J.H., 1992. Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence. MIT Press, Cambridge, MA.

[55] Hottung, A., Berto, F., Hua, C., Zepeda, N.G., Wetzel, D., Römer, M., Ye, H., Zago, D., Poli, M., Massaroli, S., Park, J., Tierney, K., 2025. Vrpagent: Llm-driven discovery of heuristic operators for vehicle routing problems. arXiv:2510.07073.

[56] Hu, T., Chen, R., Zhang, S., Yin, J., Feng, M.X., Liu, J., Zhang, S., Jiang, W., Fang, Y., Hu, S., Wang, H., Xu, Y., 2026. Controlled self-evolution for algorithmic code optimization. arXiv:2601.07348.

[57] Hu, Y., Zhou, H., 2025. Llm-powered code analysis and optimization for gaussian splatting kernels. arXiv:2509.25626.

[58] Hubert, T., Mehta, R., Sartran, L., Horváth, M.Z., Žužić, G., Wieser, E., Huang, A., Schrittwieser, J., Schroecker, Y., Masoom, H., et al., 2025. Olympiad-level formal mathematical reasoning with reinforcement learning. Nature , 1–3.

[59] Huq, S.M., Brito-Pacheco, D., Sikar, D., MOJUMDER, R., 2025. Evaluating LLMs for combinatorial optimization: One-phase and two-phase heuristics for 2d bin-packing, in: NeurIPS Workshop on Evaluating the Evolving LLM Lifecycle: Benchmarks, Emergent Abilities, and Scaling.

[60] Imajuku, Y., Horie, K., Iwata, Y., Aoki, K., Takahashi, N., Akiba, T., 2026. Ale-bench: A benchmark for long-horizon objective-driven algorithm engineering. Advances in Neural Information Processing Systems 38.

[61] Ingo, R., 1973. Evolution strategy: Optimization of technical systems by means of biological evolution. fromman-holzboog. Stuttgart 104, 15.

[62] Jain, N., Gu, A., Li, W.D., Yan, F., Zhang, T., Wang, S., Solar-Lezama, A., Sen, K., Stoica, I., 2025. Livecodebench: Holistic and contamination free evaluation of large language models for code, in: International Conference on Learning Representations, pp. 58791–58831.

[63] Jiang, J., Ding, T., Zhu, Z., 2026a. Deltaevolve: Accelerating scientific discovery through momentum-driven evolution. arXiv:2602.02919.

[64] Jiang, J., Zhu, H., Zhu, Z., 2026b. Smcevolve: Principled scientific discovery via sequential monte carlo evolution. URL: https://arxiv.org/abs/2605.15308, doi:10.48550/arXiv.2605.15308, arXiv:2605.15308.

[65] Jiang, Z., Schmidt, D., Srikanth, D., Xu, D., Kaplan, I., Jacenko, D., Wu, Y., 2025. Aide: Ai-driven exploration in the space of code. doi:10.48550/arXiv.2502.13138, arXiv:2502.13138.

[66] Karlekar, S., Zheng, C., Saebo, M., Beltran-Velez, N., Yu, S., Bowlan, J., Kucer, M., Blei, D., 2026. Duel-evolve: Reward-free test-time scaling via llm self-preferences. arXiv:2602.21585.

[67] Kasenberg, D., Castro, P.S., Eckstein, M.K., Elteto, N., Dabney, W., Wang, C.L., Engelcke, M., Mohanta, R., Dev, A., Botvinick, M.M., et al., 2026. Ai-discovered cognitive models reveal novel insights into human and animal learning. bioRxiv , 2026–05.

[68] Ke, X., Li, K., Xing, J., Zhang, Y., Cheng, J., 2026. Game-theoretic co-evolution for llm-based heuristic discovery. arXiv:2601.22896.

[69] Khrulkov, V., Galichin, A., Bashkirov, D., Vinichenko, D., Travkin, O., Alferov, R., Kuznetsov, A., Oseledets, I., 2025. Gigaevo: An open source optimization framework powered by llms and evolution algorithms. arXiv:2511.17592.

[70] Koza, J.R., 1992. Genetic programming: On the programming of computers by means of natural selection.

[71] Kravatskiy, A., Khrulkov, V., Oseledets, I., 2026. Improvevolve: Ask alphaevolve to improve the input solution and then improvise. arXiv:2602.10233.

[72] Kripner, M., 2025. Symbolic world models in lean 4 for reinforcement learning, in: Proceedings of the RLC Workshop on Programmatic Reinforcement Learning, Reinforcement Learning Journal (RLJ), Edmonton, Canada. pp. 8–18.

[73] Lalan, A., Ghosh, R., Kolsur, A., Dutta, D., 2025. A multi-agent framework for stateful inference-time search. arXiv:2510.07147.

[74] Lange, R.T., Imajuku, Y., Cetin, E., 2026. Shinkaevolve: Towards open-ended and sample-efficient program evolution, in: The Fourteenth International Conference on Learning Representations.

[75] Lee, Y., Nair, R., Zhang, Q., Lee, K., Khattab, O., Finn, C., 2026. Meta-harness: End-to-end optimization of model harnesses. URL: https://arxiv.org/abs/2603.28052, doi:10.48550/arXiv.2603.28052, arXiv:2603.28052.

[76] Lei, Y., Huang, J., Li, X., Liu, Q., Gao, L., 2026. Automatic generation of heuristic dispatching rules for dynamic hybrid flow shop scheduling via personalized multi-island reflective evolution. Tsinghua Science and Technology .

[77] Leleu, T., Gunathilaka, S., Ghimenti, F., Ganguli, S., 2026. Contrastive concept-tree search for llm-assisted algorithm discovery. arXiv:2602.03132.

[78] Li, A., Wu, C., Ge, Z., Chong, Y.H., Hou, Z., Cao, L., Ju, C., Wu, J., Li, H., Zhang, H., Feng, S., Zhao, M., Qiu, F., Yang, R., Zhang, M., Zhu, W., Sun, Y., Sun, Q., Yan, S., Liu, D., Yin, D., Shen, D., 2026a. The fm agent. arXiv:2510.26144.

[79] Li, B., Li, J., Tang, K., Yao, X., 2015. Many-objective evolutionary algorithms: A survey. ACM Computing Surveys (CSUR) 48, 1–35.

[80] Li, T., Zang, S., Münchmeyer, M., 2026b. Madevolve: Evolutionary optimization of cosmological algorithms with large language models. arXiv:2602.15951.

[81] Li, X., Sun, X., Wang, A., Li, J., Shum, C., 2026c. Cuda-l1: Improving cuda optimization via contrastive reinforcement learning. arXiv:2507.14111.

[82] Li, X., Zhou, Z., Li, Z., Yao, J., Han, B., 2026d. Learning to evolve: Scaling open-ended discovery with relative-progress rl, in: ICLR Workshop on Lifelong Agents: Learning, Aligning, Evolving.

[83] Li, Z., Schultz, J., Hennes, D., Lanctot, M., 2026e. Discovering multiagent learning algorithms with large language models. arXiv:2602.16928.

[84] Liao, G., Qin, H., Wang, Y., Golden, A., Kuchnik, M., Yetim, Y., Ang, J.J., Fu, C., He, Y., Hsia, S., Jiang, Z., Li, D., Pashkevich, U., Puvvada, V., Shi, F., Steiner, M., Xiao, R., Yan, N., Yu, X., Fang, Z., Levenstein, R., Ho, K., Zhu, H., Hammond, A., Li, R., Mathews, A., Gondkar, K., Zainul-Abedin, A., Singh, K., Yu, H., Chi, W., Huang, B., Zhang, S., Weller, N., Marine, Z., Cook, W., Wu, C.J., Liu, G., 2026. Kernelevolve: Scaling agentic kernel coding for heterogeneous ai accelerators at meta. arXiv:2512.23236.

[85] Lin, H., Ye, H., Feng, W., Huang, Q., Li, Y., Lim, H., Li, Z., Wang, X., Ma, J., Liang, Y., Zou, J., 2026. Can language models discover scaling laws? doi:10.48550/arXiv.2507.21184, arXiv:2507.21184.

[86] Liu, F., Huang, Y., Luo, S., Wang, Y., Yang, Y., Li, X., Hu, Z., Feng, J., Liu, Q., 2026a. Cognitive alpha mining via llm-driven code-based evolution. arXiv:2511.18850.

[87] Liu, F., Tong, X., Yuan, M., Lin, X., Luo, F., Wang, Z., Lu, Z., Zhang, Q., 2024. Evolution of heuristics: Towards efficient automatic algorithm design using large language model, in: Proceedings of the 41st International Conference on Machine Learning.

[88] Liu, F., Zhang, R., Lin, X., Lu, Z., Zhang, Q., 2026b. Fine-tuning large language model for automated algorithm design. arXiv:2507.10614.

[89] Liu, Q., Hao, R., Li, C., Ma, W., 2026c. Or-agent: Bridging evolutionary search and structured research for automated algorithm discovery. arXiv:2602.13769.

[90] Liu, S., Agarwal, S., Maheswaran, M., Cemri, M., Li, Z., Mang, Q., Naren, A., Boneh, E., Cheng, A., Pan, M.Z., Du, A., Keutzer, K., Cheung, A., Dimakis, A.G., Sen, K., Zaharia, M., Stoica, I., 2026d. Evox: Meta-evolution for automated discovery. arXiv:2602.23413.

[91] Liu, S., Cemri, M., Agarwal, S., Krentsel, A., Naren, A., Mang, Q., Li, Z., Gupta, A., Maheswaran, M., Cheng, A., et al., 2026e. Skydiscover: A flexible, adaptive framework for ai-driven scientific and algorithmic discovery, in: Proceedings of the ACM Conference on AI and Agentic Systems, pp. 1223–1227.

[92] Lu, C., Lu, C., Lange, R.T., Foerster, J., Clune, J., Ha, D., 2024. The ai scientist: Towards fully automated open-ended scientific discovery. doi:10.48550/arXiv.2408.06292, arXiv:2408.06292.

[93] Lupidi, A., Gauri, B., Foster, T.S., Omari, B.A., Magka, D., Pepe, A., Audran-Reiss, A., Aghamelu, M., Baldwin, N., Cipolina-Kun, L., Gagnon-Audet, J.C., Leow, C.H., Lefdal, S., Mossalam, H., Moudgil, A., Nazir, S., Tewolde, E., Urrego, I., Estape, J.A., Budhiraja, A., Chaurasia, G., Charnalia, A., Dunfield, D., Hambardzumyan, K., Izcovich, D., Josifoski, M., Mediratta, I., Niu, K., Pathak, P., Shvartsman, M., Toledo, E., Protopopov, A., Raileanu, R., Miller, A., Shavrina, T., Foerster, J., Bachrach, Y., 2026. Airs-bench: a suite of tasks for frontier ai research science agents. arXiv:2602.06855.

[94] Ma, Y.J., Liang, W., Wang, G., Huang, D.A., Bastani, O., Jayaraman, D., Zhu, Y., Fan, L., Anandkumar, A., 2024. Eureka: Human-level reward design via coding large language models. doi:10.48550/arXiv.2310.12931, arXiv:2310.12931.

[95] Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., et al., 2023. Self-refine: Iterative refinement with self-feedback. Advances in neural information processing systems 36, 46534–46594.

[96] Mang, Q., Chai, W., Li, Z., Mao, H., Zhou, S., Du, A., Li, H., Liu, S., Chen, E., Wang, Y., Chu, X., Cheng, Z., Xu, Y., Xia, T., Wang, Z., Shi, T., Yao, J., Zhao, Y., Zhang, Q., Ruan, C., Shen, Z., Liu, K., He, R., Xing, D., Li, Z., Zeng, Z., Jiang, Y., Cheng, L., Zhao, Z., Sun, Y., Zheng, W., Zhang, M., Ji, R., Tu, X., Zheng, Z., Chen, Z., Zhou, K., Wang, Z., Chen, J., Korolova, A., Henderson, P., Viswanath, P., Ganesh, V., Xie, S., Liu, Z., Song, D., Min, S., Stoica, I., Gonzalez, J.E., Shang, J., Cheung, A., 2025. Frontiercs: Evolving challenges for evolving intelligence. arXiv:2512.15699.

[97] Matai, J., Harada, T., 2026. Evorender: An agentic ai-neural rendering algorithm discovery, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 5018–5026.

[98] Min, K., Cho, K., Jang, J., Kang, S., 2026. Revolution: An evolutionary framework for rtl generation driven by large language models, in: 31st Asia and South Pacific Design Automation Conference (ASP-DAC), IEEE. pp. 282–288.

[99] Mohapatra, P., Tolkova, I., Sarkar, A., Sehgal, A., Salisbury, D.P., Von Duyke, A.L., Bouffaut, L., Klinck, H., Sun, J.J., 2026. Agentic discovery of multi-channel bioacoustic association algorithms, in: ICLR Workshop on Foundation Models for Science: Real-World Impact and Science-First Design.

[100] Nagaitsev, K., Grbcic, L., Williams, S., Iancu, C., 2026. Optimizing pytorch inference with llm-based multi-agent systems. arXiv:2511.16964.

[101] Nagda, A., Raghavan, P., Thakurta, A., 2026. Reinforced generation of combinatorial structures: Hardness of approximation. arXiv:2509.18057.

[102] Narváez, D., Battaglia, N., Fernández, A., Rossi, G., 2026. Archigenms: A neuro-symbolic evolutionary framework for greenfield microservice design, in: Anais do XXIX Congresso Ibero-Americano em Engenharia de Software, SBC, Porto Alegre, RS, Brasil. pp. 365–372. URL: https://sol.sbc.org.br/index.php/cibse/article/view/42461, doi:10.5753/cibse.2026.42461.

[103] Nian, J., Li, F., Park, D.H., Fang, Y., 2026. Rankevolve: Automating the discovery of retrieval algorithms via llm-driven evolution. arXiv:2602.16932.

[104] Nikoleit, H., Anand, A., Naredla, A.M., Röglin, H., 2025. Adversarial examples for heuristics in combinatorial optimization: An llm based approach .

[105] Novikov, A., Vũ, N., Eisenberger, M., Dupont, E., Huang, P.S., Wagner, A.Z., Shirobokov, S., Kozlovskii, B., Ruiz, F.J.R., Mehrabian, A., Kumar, M.P., See, A., Chaudhuri, S., Holland, G., Davies, A., Nowozin, S., Kohli, P., Balog, M., 2025. Alphaevolve: A coding agent for scientific and algorithmic discovery. arXiv:2506.13131.

[106] Ouyang, A., Guo, S., Arora, S., Zhang, A.L., Hu, W., Ré, C., Mirhoseini, A., 2025. Kernelbench: Can llms write efficient gpu kernels? doi:10.48550/arXiv.2502.10517, arXiv:2502.10517.

[107] Ouyang, K., Wang, H., Fang, D., 2026. Fela: A multi-agent evolutionary system for feature engineering of industrial event log data.

arXiv:2510.25223.

[108] Paglieri, D., Cross, L., Cunningham, W.A., Leibo, J.Z., Vezhnevets, A.S., 2026. Persona generators: Generating diverse synthetic personas at scale. arXiv:2602.03545.

[109] Poesia, G., Sampaio, G.G., 2025. Programmatic representation learning with language models. arXiv:2510.14825.

[110] Preintner, T., Yuan, W., König, A., Bäck, T., Raponi, E., Van Stein, N., 2025. Evocad: Evolutionary cad code generation with vision language models, in: IEEE 37th International Conference on Tools with Artificial Intelligence (ICTAI), IEEE. pp. 504–511.

[111] Qiu, J., Chen, X., Ge, L., Lin, L., Lu, Z., Zhang, Q., 2026a. Evolving interdependent operators with large language models for multi-objective combinatorial optimization. arXiv:2601.17899.

[112] Qiu, J., Zhuang, H., Liu, F., Liu, J., Zhang, Q., 2026b. Evodr: Evolving dispatching rules via large language model for dynamic flexible assembly flow shop scheduling. arXiv:2601.15738.

[113] Ran, N., Li, Z., Wang, Y., Ran, Q., Zhang, X., Feng, S., Allmendinger, R., Zhao, X., 2025. Mcce: A framework for multi-llm collaborative co-evolution. arXiv:2510.06270.

[114] Ray, P., Brahma, P.P., Liu, Z., Barsoum, E., 2026. Adaptevolve: Improving efficiency of evolutionary ai agents through adaptive model selection. arXiv:2602.11931.

[115] Romera-Paredes, B., Barekatain, M., Novikov, A., Balog, M., Kumar, M.P., Dupont, E., Ruiz, F.J., Ellenberg, J.S., Wang, P., Fawzi, O., et al., 2024. Mathematical discoveries from program search with large language models.

[116] Sadikov, A., 2026. LLM-guided evolutionary program synthesis for quasi-monte carlo design, in: The Fourteenth International Conference on Learning Representations.

[117] Saketos, V., Kaltenbach, S., Litvinov, S., Koumoutsakos, P., 2025. Data-driven discovery of interpretable kalman filter variants through large language models and genetic programming. arXiv:2508.11703.

[118] Sapora, S., Hjelm, D., Toshev, A., Attia, O., Mazoure, B., 2026. Grace: A language model framework for explainable inverse reinforcement learning. arXiv:2510.02180.

[119] Shi, Z., Wang, J., Cheng, T., Xu, C., Qian, W., Xu, Q., 2026. Cktevo: Repository-level rtl code benchmark for design evolution. arXiv:2603.08718.

[120] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S., 2023. Reflexion: language agents with verbal reinforcement learning, in: Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems, NeurIPS.

[121] Song, Z.Y., Cai, Z., Zhang, S., Wei, J., Pan, J., Qiu, S., Cao, Q.H., Hou, T.J., Liu, X., xing Luo, M., Zhu, H.X., 2025. Iterated agent for symbolic regression. arXiv:2510.08317.

[122] Storn, R., Price, K., 1997. Differential evolution–a simple and efficient heuristic for global optimization over continuous spaces. Journal of Global Optimization 11, 341–359. doi:10.1023/A:1008202819921.

[123] Su, H., Zheng, Y., Li, Y., 2026. Contextevolve: Multi-agent context compression for systems code optimization. arXiv:2602.02597.

[124] Sun, J., Chong, W., Tu, S., Zhang, Q., Zhang, Y., Chai, J., Wang, X., Lin, W., Yin, G., Zhao, D., 2026. Autosearch: Adaptive search depth for efficient agentic rag via reinforcement learning. doi:10.48550/arXiv.2604.17337, arXiv:2604.17337.

[125] Sun, Y., Ye, F., Chen, Z., Wei, K., Cai, S., 2025. Automatically discovering heuristics in a complex sat solver with large language models. arXiv:2507.22876.

[126] Tanugula, R., Chintapalli, D., Chandra, S., 2025. Lark: Biologically inspired neuroevolution for multi-stakeholder LLM agents, in: NeurIPS Workshop on Efficient Reasoning.

[127] Telikani, A., Tahmassebi, A., Banzhaf, W., Gandomi, A.H., 2021. Evolutionary machine learning: A survey. ACM Computing Surveys (CSUR) 54, 1–35.

[128] Tsoukalas, G., Saha, R., Thakur, A., Reguyal, S., Chaudhuri, S., 2026. Learning interestingness in automated mathematical theory formation, in: The Thirty-ninth Annual Conference on Neural Information Processing Systems.

[129] Tu, S., Lin, J., Tian, X., Zhang, Q., Li, L., Fu, Y., Xu, N., He, W., Lan, X., Jiang, D., Zhao, D., 2025. Enhancing LLM reasoning with iterative DPO: A comprehensive empirical investigation, in: Second Conference on Language Modeling.

[130] Tu, S., Lin, J., Zhang, Q., Tian, X., Li, L., Lan, X., Zhao, D., 2026a. Learning when to think: Shaping adaptive reasoning in r1-style models via multi-stage RL, in: The Thirty-ninth Annual Conference on Neural Information Processing Systems.

[131] Tu, S., Ma, Y., Lin, J., Zhang, Q., Lan, X., Li, J., Xu, N., Li, L., Zhao, D., 2026b. Paperaudit-bench: Benchmarking error detection in research papers for critical automated peer review. doi:10.48550/arXiv.2601.19916, arXiv:2601.19916.

[132] Venkatraman, S., Jain, V., Mittal, S., Shah, V., Obando-Ceron, J.S., Bengio, Y., Bartoldson, B.R., Kailkhura, B., Lajoie, G., Berseth, G., Malkin, N., Jain, M., 2025. Recursive self-aggregation unlocks deep thinking in large language models. CoRR abs/2509.26626. arXiv:2509.26626.

[133] Wan, C., Dai, X., Wang, Z., Li, M., Wang, Y., Mao, Y., Lan, Y., Xiao, Z., 2025. Loongflow: Directed evolutionary search via a cognitive plan-execute-summarize paradigm. arXiv:2512.24077.

[134] Wang, C., Ma, S., Ma, Z., Gong, Y.J., 2026a. Evolution of benchmark: Black-box optimization benchmark design through large language model. arXiv:2601.21877.

[135] Wang, H., Wu, Y., Chang, D., Wei, L., Heldt, L., 2026b. Self-evolving recommendation system: End-to-end autonomous model optimization with llm agents. arXiv:2602.10226.

[136] Wang, L., Duan, P., Wang, H., Wang, Y., Xu, J., Zheng, N., Ma, Z., 2026c. Evolvesignal: A large language model powered coding agent for discovering traffic signal control strategies. arXiv:2509.03335.

[137] Wang, X., Chen, J., Farhang, A.R., Stiles, S., Horstmann, K.A., Sehgal, A., Light, J., Van Valen, D., Yue, Y., Sun, J.J., 2025a. Agentic superoptimization of bioimaging analysis workflows, in: LLM for Scientific Discovery: Reasoning, Assistance, and Collaboration.

[138] Wang, X., Wei, J., Schuurmans, D., Le, Q.V., Chi, E.H., Narang, S., Chowdhery, A., Zhou, D., 2023. Self-consistency improves chain of thought reasoning in language models, in: The Eleventh International Conference on Learning Representations, ICLR.

[139] Wang, Y., Liu, Q., Wang, Z., Li, Z., Wei, W., Liu, Y., Bao, Y., 2025b. Promptbridge: Cross-model prompt transfer for large language models. arXiv:2512.01420.

[140] Wang, Y., Su, S.R., Zeng, Z., Xu, E., Ren, L., Yang, X., Huang, Z., He, X., Ma, L., Peng, B., Cheng, H., He, P., Chen, W., Wang, S., Du, S.S., Shen, Y., 2025c. Thetaevolve: Test-time learning on open problems. arXiv:2511.23473.

[141] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E.H., Le, Q.V., Zhou, D., 2022. Chain-of-thought prompting elicits reasoning in large language models, in: Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems, NeurIPS.

[142] Wu, C., Chen, H., Yuan, C., Yang, Q., Song, G., Gao, Y., Luo, L., Chen, F.Y., Guo, M., 2026. Agentltv: An agent-based unified search-and-evolution framework for automated lifetime value prediction. arXiv:2602.21634.

[143] Wu, X., Wu, S.h., Wu, J., Feng, L., Tan, K.C., 2024. Evolutionary computation in the era of large language model: Survey and roadmap. IEEE Transactions on Evolutionary Computation 29, 534–554.

[144] Xia, S., Sun, Y., Liu, P., 2026. SR-scientist: Scientific equation discovery with agentic AI, in: The Fourteenth International Conference on Learning Representations.

[145] Xiang, C., Wei, Y., Ma, J., Wang, H., Yan, J., 2026. Beam: Bi-level memory-adaptive algorithmic evolution for llm-powered heuristic design. URL: https://arxiv.org/abs/2604.12898, doi:10.48550/arXiv.2604.12898, arXiv:2604.12898.

[146] Xiao, H., Hui, N., Xu, Y., Li, Z., Peng, J., 2026. Medical multimodal large language models: A survey. Information Fusion 134, 104386. doi:10.1016/j.inffus.2026.104386.

[147] Xu, K., Chai, J., Li, S., Fu, Y., Zhu, Y., Zhao, D., 2025. Dipllm: Fine-tuning LLM for strategic decision-making in diplomacy, in: Forty-second International Conference on Machine Learning, ICML.

[148] Xu, L., Zhang, X., Tuo, Q., Li, R., 2026. Sea-ts: Self-evolving agent for autonomous code generation of time series forecasting algorithms. arXiv:2603.04873.

[149] Yan, M., Peng, B., Coleman, B., Chen, Z., Xie, Z., Chen, S., He, Z., Sachdeva, N., Ye, I., Wang, W., Wang, C., Chi, E.H., Pereira, F., Kang, W.C., Cheng, D.Z., Wang, B., 2026. PACEvolve: Enabling long-horizon progress-aware consistent evolution, in: ICLR Workshop on Lifelong Agents: Learning, Aligning, Evolving.

[150] Yang, C., Wang, X., Lu, Y., Liu, H., Le, Q.V., Zhou, D., Chen, X., 2023. Large language models as optimizers, in: The Twelfth International Conference on Learning Representations.

[151] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T.L., Cao, Y., Narasimhan, K., 2023a. Tree of thoughts: Deliberate problem solving with large language models. doi:10.48550/arXiv.2305.10601, arXiv:2305.10601.

[152] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y., 2023b. React: Synergizing reasoning and acting in language models. doi:10.48550/arXiv.2210.03629, arXiv:2210.03629.

[153] Ye, H., Wang, J., Cao, Z., Berto, F., Hua, C., Kim, H., Park, J., Song, G., 2024. Reevo: Large language models as hyper-heuristics with reflective evolution, in: Advances in Neural Information Processing Systems.

[154] Yu, C., Liang, R., Ho, C.T., Ren, H., 2025. Autonomous code evolution meets np-completeness. arXiv:2509.07367.

[155] Yu, J., Sun, Y., 2026. Writeer: A multi-objective, preference-driven multi-agent framework for human-like advanced text generation, in: Proceedings of the AAAI Conference on Artificial Intelligence, pp. 29856–29864.

[156] Yu, Z., Feng, K., Zhao, Y., He, S., Zhang, X.P., Cohan, A., 2026. Alpharesearch: Accelerating new algorithm discovery with language models. arXiv:2511.08522.

[157] Yu, Z., Yang, L., 2026. Evolutionary ensemble of agents. URL: https://arxiv.org/abs/2605.09018, doi:10.48550/arXiv.2605.09018, arXiv:2605.09018.

[158] Yuksel, K.A., Sawaf, H., 2025. Evorisk: Autonomously discovered regime-adaptive resilience-aware financial metric. Available at SSRN 5791002 .

[159] Yun, J., Lee, H.J., Jeon, I., 2025. Quantevolve: Automating quantitative strategy discovery through multi-agent evolutionary framework. arXiv:2510.18569.

[160] Zhai, Y., Wei, Z., Li, R., Pan, K., Liu, S., Zhang, L., Ji, J., Zhang, W., Zhang, Y., Zhang, Y., 2025. x-evolve: Solution space evolution powered by large language models. arXiv:2508.07932.

[161] Zhan, Z.H., Shi, L., Tan, K.C., Zhang, J., 2022. A survey on evolutionary computation for complex continuous optimization. Artificial Intelligence Review 55, 59–110.

[162] Zhang, C., Cortiñas, R.G., Karamlou, A.H., Noll, N., Provazza, J., Bausch, J., Shirobokov, S., White, A., Claassen, M., Kang, S.H., Senior, A.W., Tomašev, N., Gross, J., Lee, K., Schuster, T., Huggins, W.J., Celik, H., Greene, A., Kozlovskii, B., Heras, F.J.H., Bengtsson, A., Dau, A.G., Drozdov, I., Ying, B., Livingstone, W., Sivak, V., Yosri, N., Quintana, C., Abanin, D., Abbas, A., Acharya, R., Beni, L.A., Aigeldinger, G., Alcaraz, R., Alcaraz, S., Andersen, T.I., Ansmann, M., Arute, F., Arya, K., Askew, W., Astrakhantsev, N., Atalaya, J., Ballard, B., Bardin, J.C., Bates, H., Karimi, M.B., Bilmes, A., Bilodeau, S., Borjans, F., Bourassa, A., Bovaird, J., Bowers, D., Brill, L., Brooks, P., Broughton, M., Browne, D.A., Buchea, B., Buckley, B.B., Burger, T., Burkett, B., Busnaina, J., Bushnell, N., Cabrera, A., Campero, J., Chang, H.S., Chen, S., Chen, Z., Chiaro, B., Chih, L.Y., Cleland, A.Y., Cochrane, B., Cockrell, M., Cogan, J., Collins, R., Conner, P., Cook, H., Courtney, W., Crook, A.L., Curtin, B., Das, S., Damyanov, M., Debroy, D.M., Lorenzo, L.D., Demura, S., Rose, L.B.D., Paolo, A.D., Donohoe, P., Dunsworth, A., Ehimhen, V., Eickbusch, A., Elbag, A.M., Ella, L., Elzouka, M., Enriquez, D., Erickson, C., Ferreira, V.S., Flores, M., Burgos, L.F., Forati, E., Ford, J., Fowler, A.G., Foxen, B., Fukami, M., Fung, A.W.L., Fuste, L., Ganjam, S., Garcia, G., Garrick, C., Gasca, R., Gehring, H., Geiger, R., Genois, É., Giang, W., Gidney, C., Gilboa, D., Goeders, J.E., Gonzales, E.C., Gosula, R., de Graaf, S.J., Graumann, D., Grebel, J., Guerrero, J., Guimarães, J.D., Ha, T., Habegger, S., Hadick, T., Hadjikhani, A., Harrigan, M.P., Harrington, S.D., Hartshorn, J., Heslin, S., Heu, P., Higgott, O., Hiltermann, R., Hilton, J., Huang, H.Y., Hucka, M., Hudspeth, C., Huff, A., Jeffrey, E., Jevons, S., Jiang, Z., Jin, X., Joshi, C., Juhas, P., Kabel, A., Kang, H., Kang, K., Kaufman, R., Kechedzhi, K., Khattar, T., Khezri, M., Kim, S., King, R., Kiss, O., Klimov, P.V., Knaut, C.M., Kobrin, B., Kostritsa, F., Kreikebaum, J.M., Kudo, R., Kueffler, B., Kumar, A., Kurilovich, V.D., Kutsko, V., Lacroix, N., Landhuis, D., Lange-Dei, T., Langley, B.W., Laptev, P., Lau, K.M., Guevel, L.L., Ledford, J., Lee, J., Lester, B.J., Leung, W., Li, L., Li, W.Y., Li, M., Lill, A.T., Lloyd, M.T., Locharla, A., Lundahl, D., Lunt, A., Madhuk, S., Maiti, A., Maloney, A., Mandra, S., Martin, L.S., Martin, O., Mascot, E., Das, P.M., Maslov, D., Mathews, M., Maxfield, C., McClean, J.R., McEwen, M., Meeks, S., Miao, K.C., Molavi, R., Molina, S., Montazeri, S., Neill, C., Newman, M., Nguyen, A., Nguyen, M., Ni, C.H., Niu, M.Y., Oas, L., Orosco, R., Ottosson, K., Pagano, A., Peek, S., Peterson, D., Pizzuto, A., Portoles, E., Potter, R., Pritchard, O., Qian, M., Ranadive, A., Reagor, M.J., Resnick, R., Rhodes, D.M., Riley, D., Roberts, G., Rodriguez, R., Ropes, E., Rosenberg, E., Rosenfeld, E., Rosenstock, D., Rossi, E., Rower, D.A., Rudolph, M.S., Salazar, R., Sankaragomathi, K., Sarihan, M.C., Satzinger, K.J., Schaefer, M., Schroeder, S., Schurkus, H.F., Shahingohar, A., Shearn, M.J., Shorter, A., Shutty, N., Shvarts, V., Small, S., Smith, W.C., Sobel, D.A., Somma, R.D., Spells, B., Springer, S., Sterling, G., Suchard, J., Szasz, A., Sztein, A., Taylor, M., Thiruraman, J.P., Thor, D., Timucin, D., Tomita, E., Torres, A., Torunbalci, M.M., Tran, H., Vaishnav, A., Vargas, J., Vdovichev, S., Vidal, G., Heidweiller, C.V., Voorhees, M., Waltman, S., Waltz, J., Wang, S.X., Ware, B., Watson, J.D., Wei, Y., Weidel, T., White, T., Wong, K., Woo, B.W.K., Wood, C.J., Woodson, M., Xing, C., Yao, Z.J., Yeh, P., Yoo, J., Young, E., Young, G., Zalcman, A., Zhang, R., Zhang, Y., Zhu, N., Zobrist, N., Zou, Z., Bortoli, G., Boixo, S., Chen, J., Chen, Y., Devoret, M., Hansen, M., Jones, C., Kelly, J., Kohli, P., Korotkov, A., Lucero, E., Manyika, J., Matias, Y., Megrant, A., Neven, H., Oliver, W.D., Ramachandran, G., Babbush, R., Smelyanskiy, V., Roushan, P., Kafri, D., Sarpong, R., Berry, D.W., Ramanathan, C., Mi, X., Bengs, C., Ajoy, A., Minev, Z.K., Rubin, N.C., O'Brien, T.E., 2025a. Quantum computation of molecular geometry via many-body nuclear spin echoes. arXiv:2510.19550.

[163] Zhang, J., Zhao, B., Yang, W., Foerster, J., Clune, J., Jiang, M., Devlin, S., Shavrina, T., 2026a. Hyperagents. doi:10.48550/arXiv.2603.19461, arXiv:2603.19461.

[164] Zhang, W., Zong, Y., Yue, Y., Yuan, K., 2026b. Theory-driven modeling and llm-guided evolution for power system scheduling, in: ICLR Workshop on AI with Recursive Self-Improvement.

[165] Zhang, Y., Cheng, R., Yi, G., Tan, K.C., 2026c. A systematic survey on large language models for evolutionary optimization: From modeling to solving. URL: https://arxiv.org/abs/2509.08269, doi:10.48550/arXiv.2509.08269, arXiv:2509.08269.

[166] Zhang, Y., Duan, Y., Zhang, Z., He, J., Zheng, S., 2025b. Population-evolve: a parallel sampling and evolutionary method for llm math reasoning. arXiv:2512.19081.

[167] Zhang, Z., Liao, Y., Chen, C., He, J., Wu, D., Yu, Q., Gao, Y., Yang, J., Zhang, K., Chng, E.S., Zhong, X., 2026d. Textbfgs: A case-based reasoning approach to code optimization via error-operator retrieval. doi:10.48550/arXiv.2602.00059, arXiv:2602.00059.

[168] Zhao, L., Huang, W., Guo, Y., Bian, J., Wang, C., Zhang, X., 2026. Large language model-powered evolutionary code optimization on a phylogenetic tree. arXiv:2601.14523.

[169] Zheng, J., Zhang, J., Luo, Y., Mao, Y., Gao, Y., Du, L., Chen, H., Zhang, N., 2026. Can we predict before executing machine learning agents?, in: ICLR Workshop on Lifelong Agents: Learning, Aligning, Evolving.

[170] Zheng, Y., Xing, Z., Zhang, Q., Jin, B., Li, P., Zheng, Y., Xia, Z., Zhan, K., Lang, X., Chen, Y., Zhao, D., 2024. Planagent: A multi-modal large language agent for closed-loop vehicle motion planning. doi:10.48550/arXiv.2406.01587, arXiv:2406.01587.

[171] Zhou, Z., Cao, C., Feng, X., Li, X., Li, Z., Lu, X., Yao, J., Huang, W., Cheng, T., Zhang, J., Jiang, T., Xu, L., Zheng, Y., Miranda, B., Liu, T., Koyejo, S., Sugiyama, M., Han, B., 2026. Alphaapollo: A system for deep agentic reasoning. doi:10.48550/arXiv.2510.06261, arXiv:2510.06261.

[172] Zou, Q., Lam, H.H., Zhao, W., Tang, Y., Chen, T., Yu, S., Zhang, T., Liu, C., Ji, X., Liu, D., 2026. Fml-bench: Benchmarking machine learning agents for scientific research. arXiv:2510.10472.